

Jira Enhancer: A Governance-Aware Human-AI Collaboration System for Epic-to-Story Decomposition in Software Engineering

MANISH KUMAR AGRAWAL, Oracle India Pvt. Ltd., India and Indian Institute of Technology Roorkee, India

SANDEEP KUMAR, Indian Institute of Technology Roorkee, India

ASHOK SHUKLA, Oracle Corps, United States

SUBHAM KUMAR, Oracle India Pvt. Ltd., India

ABHISHEK GUPTA, Oracle India Pvt. Ltd., India

NANDAGOPAL SRINIVASAN, Oracle India Pvt. Ltd., India

Human-AI collaboration in software engineering now comprises coding, requirements enhancement, planning, and controlled updates to collaborative engineering artifacts. We introduce Jira Enhancer, which converts sparsely specified epics into reviewable stories while leaving human reviewers in control of issue-tracker updates. The method is a combination of evidence gathering, decomposition, enrichment with confidence, Thompson sampling based prompting, approval, and safe writeback. Decomposition entails two conditions: machine confidence and human approval. While there exists a Laplace domain model that captures the dynamics of confidence lag, it cannot substitute for iterative refinement observation. We evaluate authenticated evidence from the field, artifact evaluation blind to the evidence, matched baseline comparison, controlled fault injection, offline replay, and sensitivity generation analysis individually. In two deployment phases, we counted 70 story endpoints: there were 69 stories developed, while duplicate-safe check revealed one pre-existing story. The mean displayed confidence C_{10} improved from 7.46 to 9.63 with a mean difference of 2.17 points (95% bootstrap confidence interval [1.94, 2.40]). In another matched study, five reviewers independently evaluated 42 anonymized software artifacts created using 14 common inputs by a deterministic template/rule baseline (B1), one shot drafting control (B2), and Jira Enhancer (B3). B3 exceeded B1 by 1.56 points (95% epic-bootstrap CI [1.23, 1.91], Holm-adjusted exact $p = 0.00024$). Compared with B2, B3 reduced reviewer-recorded assessment time by 1.02 minutes (95% CI [-1.57, -0.48], $p = 0.0098$) and received Ready-or-Minor-revision ratings in 88.1% rather than 66.7% of primary evaluations. B3 achieved a higher blinded story-package quality score than B2 (0.10, 95% confidence interval [-0.13, 0.36], $p = 0.485$); this test is not sensitive to the governance or writeback attributes of B3. In an independent fault injection test, B3 source bounded transaction envelope prevented all injected unsafe mutations and permitted all eligible writes. All together, these results confirm that B3 is the better end-to-end governed workflow compared to B1 and B2 under these test conditions. They do not establish downstream business impact, compute efficiency, delivery calibration, or cross-domain generalization.

CCS Concepts: • **Software and its engineering** → **Requirements analysis**; • **Human-centered computing** → **Collaborative interaction**; • **Computing methodologies** → *Reinforcement learning*.

Additional Key Words and Phrases: Human-AI collaboration, requirements engineering, agile planning, agentic software engineering, human-in-the-loop AI, reinforcement learning, governance, confidence calibration

Authors' addresses: Manish Kumar Agrawal, Oracle India Pvt. Ltd., Bengaluru, India, Indian Institute of Technology Roorkee, Roorkee, India, m_agrawal@cs.iitr.ac.in; Sandeep Kumar, Indian Institute of Technology Roorkee, Roorkee, India, sandeep.kumar@cs.iitr.ac.in; Ashok Shukla, Oracle Corps, California, United States, ashok.shukla@oracle.com; Subham Kumar, Oracle India Pvt. Ltd., Bengaluru, India, subham.su.kumar@oracle.com; Abhishek Gupta, Oracle India Pvt. Ltd., Bengaluru, India, abhishek.ae.gupta@oracle.com; Nandagopal Srinivasan, Oracle India Pvt. Ltd., Bengaluru, India, nandagopal.srinivasan@oracle.com.

2026. Manuscript submitted to ACM

Manuscript submitted to ACM

ACM Reference Format:

Manish Kumar Agrawal, Sandeep Kumar, Ashok Shukla, Subham Kumar, Abhishek Gupta, and Nandagopal Srinivasan. 2026. Jira Enhancer: A Governance-Aware Human-AI Collaboration System for Epic-to-Story Decomposition in Software Engineering. *ACM Trans. Softw. Eng. Methodol.* 1, 1 (August 2026), 46 pages.

1 INTRODUCTION

The topic of this research paper addresses the problem of human-AI cooperation in software engineering: how AI agents can transform incomplete planning intent into reviewable and execution-ready Jira stories while maintaining accountability, governance, and control over shared engineering records. The problem under consideration is associated with well-known requirements-engineering issues, including ambiguity, incompleteness, and stakeholder coordination [44, 46]. It is also an operational problem because teams do not need another channel that produces fluent text which must then be reconciled manually. They need support that fits how planning decisions are reviewed, challenged, approved, and audited.

This problem involves machine reasoning, requirements engineering, software delivery operations, and enterprise governance. It is particularly relevant to organizations that want AI assistance but do not accept opaque automation, uncontrolled writeback, or weak accountability.

Recent software-engineering roadmaps also describe agentic systems as collaborators across the full development process, rather than only as code generators, and they stress trust, human values, and verifiable evaluation [1, 22, 49].

We therefore treat Jira Enhancer as an AI-supported collaborative planning workflow rather than as a generator. The AI provides contextual information, decomposition suggestions, confidence estimates, and draft revisions. Human reviewers evaluate the evidence, approve or decline proposed stories, and retain ultimate control over external issue-tracker mutation. Such a framing is consistent with human-AI interaction literature that recommends systems reveal their capabilities, uncertainty, and user control at the point of decision [2]. It also matches conclusions from studies of automation trust that emphasize the importance of appropriate rather than blind reliance on AI [33, 48].

1.1 Research Scope and Research Questions

AI-supported planning is considered not as one-shot automation but as a joint socio-technical workflow. The object of study is the collaboration boundary: which stages of epic decomposition can be delegated to an AI agent, which stages require human decision-making, how confidence and evidence should be represented, and how shared software-engineering records can be changed without reducing accountability.

Therefore, we investigate four research questions:

RQ1: How can a human-AI workflow split underspecified software epics into reviewable implementation stories without compromising human authority over acceptance and writeback?

RQ2: How can machine confidence, evidence provenance, and policy gates be combined to help reviewers determine whether AI-generated planning records are reliable enough for action?

RQ3: What failure modes emerge when agentic planning systems interact with enterprise issue trackers, including missing evidence, duplicate story creation, partial writeback failure, and stale approvals?

RQ4: How does a governed human-AI workflow perform, relative to executable template/rule and generic-LLM baselines, on decomposition quality, review effort, and implementation readiness?

The questions consider effective collaboration, joint performance, persistent agents, and new failure modes in software-engineering workflows. The contribution extends beyond implementation of the tool. It presents an executable study of how AI agents and human reviewers coordinate around real software-engineering records under explicit governance constraints.

2 LITERATURE REVIEW AND LIMITATIONS

In modern software development, applications depend on an issue-tracking tool, such as Jira, to support execution. Although teams can capture high-level epics, these epics may be underspecified because they lack acceptance criteria, dependencies, and uniform estimation granularity. Previous work on agile requirements [13, 24], requirements standards [25], and issue-reporting quality [10] shows that structure, completeness, and actionability greatly influence subsequent engineering processes [41]. Modern tools usually come in one of three forms. They are either:

- static templates with no intelligence,
- recommendation tools with no collaboration semantics or governance (high risk), or
- unable to correlate confidence, review decisions, and write permissions (low trust).

This shortcoming is observed in the everyday practice when doing simple things such as re-writing tickets during sprint planning, inferring hidden dependencies from distributed documentation by engineering leads, and dealing with unclear acceptance criteria by quality assurance during testing. The shortcoming is not limited to low process efficiency but rather to the inability to shape the information between capturing intentions and implementation. Similar issues were identified in the bug reporting and issue triage research with regards to vague terminology and poor routing signals as factors increasing coordination cost [3, 26, 32]. Thus, the technical problem is to build a reliable machine system that can:

- (1) infer missing stories from an epic,
- (2) fill out stories until they become ready for implementation,
- (3) estimate confidence and uncertainty,
- (4) approve and enforce policies,
- (5) write results deterministically and auditably.

As a system designed to work in the real world has to deal with partial evidence, limited access to external systems, and multiple users editing things simultaneously, it should be able to run reliably both with clean data and with the enterprise noise.

2.1 Background from Existing Systems

The proposed solution will help solve shortcomings of the related systems of three classes. First, static authoring templates in project-management software offer better formatting but do not significantly enhance requirements completeness and decomposition quality in general. Second, advanced large language model copilots generate fluent texts but frequently lack formal confidence calibration, policy coupling, and conflict-safe mutation policies. Finally, workflow automation engines can deal with deterministic state transitions but rarely adopt prompt strategies or build stories out of evidence. These systems alone cannot help humans and AI cooperate in order to share context, manage uncertainty, recover from failure, and retain responsibility for co-created software-engineering records.

LLM and generative AI literature in software engineering and requirements engineering show good results for requirements engineering, issue resolution, and development workflows [14, 17]. At the same time, there are challenges

in reliability, controllability, and evaluations of the existing approaches [14, 17]. Research on code generation assistants describes conditions showing a similar pattern: developers inspect, revise, and place generated outputs within the task context rather than accepting them unchanged [7, 59]. In case of agile planning, decomposition quality and review effort determine execution predictability, but governance-aware human oversight is still required for trustworthy deployments [2, 41, 57].

Multi-armed bandits and reinforcement learning algorithms are popular methods for selecting policies in an online setting under uncertainty [5, 35, 58]. Application of these algorithms to enterprise planning is difficult due to delayed and potentially unreliable rewards and the need to adhere to governance constraints. In turn, the suggested approach solves this problem by coupling an adaptive prompt policy to an auditable workflow state and explicit approval gates.

Transaction safety [9] and deterministic workflow control [60] are important aspects of writeback safety in LLM-based code generation. They are considered in LLM-based retrieval and tool-usage pipelines. Combining these aspects with RL-guided, LLM-based decomposition is intended to increase planning intelligence while preserving mutation safety in shared issue trackers.

Thus, the proposed approach cannot be considered solely a code generation advancement but rather its unique combination of search, decomposition, confidence calibration, policy evaluation, approval semantics, and safe writeback.

2.2 Related Work: Thematic Positioning

Theme A: Evidence retrieval and context construction. Studies on retrieval-augmented generation and dense retrieval [34, 53] show that evidence quality and the retrieval process are essential elements for response quality and hallucinations reduction [4, 21, 30, 38]. Jira Enhancer incorporates this insight in the form of context packing: rather than delivering evidence blocks to the generator, Jira Enhancer ranks the evidence and maps missing signals.

Theme B: Quality of reasoning via bounded refinement. Studies on reasoning-prompting and self-reflection [55, 62] show that it has positive impact when models are capable of deliberating, evaluating their outputs, and refining them according to some explicit criteria [19, 42, 67]. Jira Enhancer follows this idea with a rubric-based critic and a bounded number of retries. In other words, improvements happen within explicitly stated scoring dimensions rather than limitless prompting.

Theme C: Calibration, alignment, and reliable deployment. Alignment research highlights the fact that fluent output is not a sufficient condition for reliable performance in a critical workflow environment without calibration and human intervention [27, 36, 47]. Moreover, research on human-AI interaction and automation emphasizes the importance of adequate trust, mental models, right level of automation, and human control [2, 6, 33, 48]. Jira Enhancer resolves this issue by tying confidence to the policy gates and requiring approval before performing any ticket mutation.

Theme D: Constrained tools and safe execution. Studies on LLM tool use [68] demonstrate that models become more reliable when their outputs go through a constrained interface and verifiable tool steps [19]. Jira Enhancer implements this approach with deterministic writeback, which includes hashing, idempotency, and limited field modifications.

3 PROPOSED METHODOLOGY

The closed loop is made up of five layers: (1) scope resolution maps issue, sprint, epic, or release scopes to issue sets; (2) semantic enrichment develops structured descriptions, acceptance criteria, risks, dependencies, and next steps; (3) the RL layer selects prompt packs using Thompson Sampling and records confidence trajectories; (4) policy and

Table 1. Comparison of the main paradigms discussed in the background and related work: representative strengths, enterprise-planning limitations, and the position of Jira Enhancer (B3) relative to B0/B1/B2.

Paradigm	Representative prior work	Typical strength	Common constraint in enterprise planning	Position of B0/B1/B2/B3
Template/rule decomposition	RE standards, agile requirements, and user-story quality work [13, 25, 41]	High consistency and stable formatting	Little ability to adapt to ambiguous epics; poor semantic understanding of unfamiliar situations	B0 and B1 are closest relatives of this family; B3 retains its structure but adds adaptive semantic reasoning
Generic LLM drafting	Prompt-driven generation, code assistants, and reasoning methods [7, 12, 17, 62]	natural-language drafts and broad transfer between tasks	Fluency, but lack of structure; confidence is uncoupled from mutation authority	B2 is closest relative of this family; B3 adds critic loops, validators and governance coupling
Retrieval-augmented generation	Retrieval-augmented and dense-retrieval methods [4, 29, 34]	Strong evidence base for grounding and factual memory recall	Varying retrieval quality; context-packing is not well defined for workflow-mutation situations	B3 converts retrieval into operation through ranking of context packets and explicit tracking of missing signals
Agent/tool orchestration	Tool-use methods and agent benchmarks [39, 40, 54, 66]	Strong multi-step execution ability and coordination with external systems	Inconsistent run-time behavior and risk of harm [50] when there is no deterministic bound on mutation	B3 constrains execution using policy-gated, hash-checked and idempotent write-back
Trust [33], calibration [20], governance	Alignment, human-AI collaboration, automation-trust, and socio-technical SE studies [6, 47, 48, 57]	Facilitates control and accountability [2]	Typically investigated in settings away from issue-tracker mutation	B3 directly implements this layer in Jira using human approval and policy gates

governance apply thresholds, flags, and human-approval rules; and (5) writeback and audit perform bounded mutations with conflict checks and immutable traces.

First, the system generates stories, refines them toward a displayed confidence target (for example, $C_{10} \geq 9$), and creates a candidate story only after individual approval through controlled field mappings. The AI retrieves evidence, drafts and scores candidate stories, and performs bounded refinement. The reviewer evaluates the context and risk but retains final control. Generation, review, and writeback share a common state rather than being distinct processes, consistent with socio-technical guidelines for technology assessment in context [56, 57]. The candidate is a stateful planning artifact and not a throwaway answer. The log contains the evidence, the selected prompt arm, the variation in confidence, the intervention of the reviewer, and the policy choices. Creative generation can be reattempted, yet any mutation is irreversible and constrained by the policy check for conflicts. This allows normal structuring to continue while preserving auditable human agency for significant mutations.

3.1 System Architecture

Figure 1 presents the complete system architecture, showing how the LLM, RL-guided prompt-selection layer, and human-governance mechanisms interact to address the software-engineering and requirements-engineering challenges discussed above. The subsequent sections explain the main architectural components in greater detail.

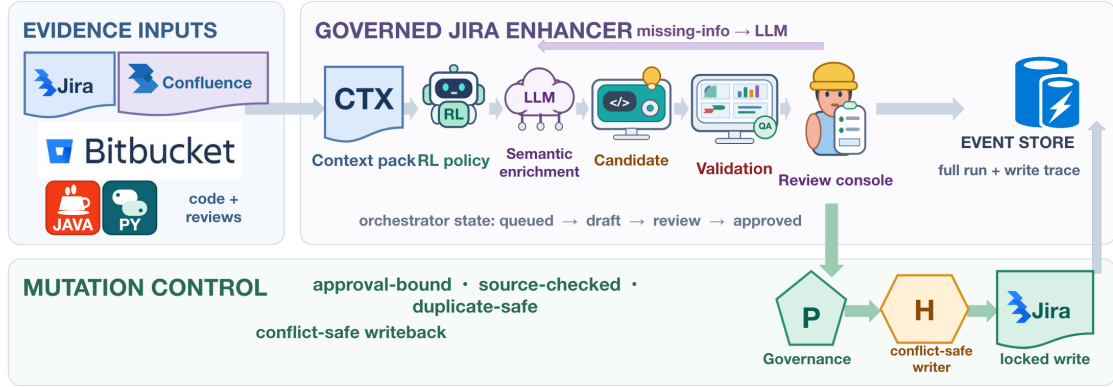


Fig. 1. Human-AI collaboration architecture. The Jira, Confluence, and Bitbucket provide evidence in the form of issues, requirements, design, code, and reviews. The orchestrator controls evidence-based story creation, bounded prompt arm selection, governance checks, audit storage, and controlled writeback of Jira issues. The tags define the sequence of runtime contracts that starts from the context extraction and human actions and proceeds with scoring, policy check, approved candidate generation, and mutation.

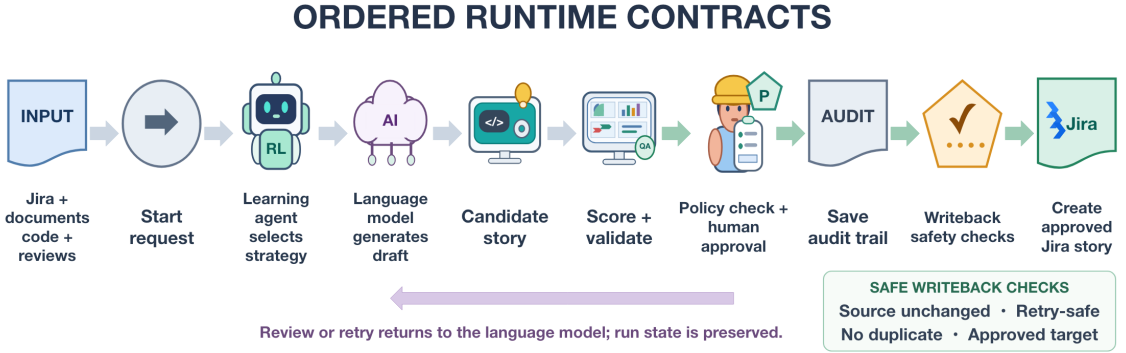


Fig. 2. Sequence of the runtime contracts according to Fig. 1. The Jira, requirements, code, and review input is fed into the orchestrator. Safe prompt selection and scoring generate the candidates, policy check and tracing storage precede the creation of approved candidates; writeback is the last step of the process.

3.1.1 *Reading Figures 1–3.* The set of Figures 1–3 provides three perspectives: architecture and trust boundaries, operational sequence, and internal decomposition of agents. The deliberate overlap helps to conduct separate reviews of completeness of the architecture, correctness of the execution, and responsibilities of agents.

Figure 1: system architecture overview. Figure 1 places Jira epics, stories, and writeback targets; Confluence requirements and design evidence; and Bitbucket code and review signals outside the AI boundary. Inside Jira Enhancer, the review console, orchestrator, semantic enrichment engine, RL policy, governance layer, event store, and conflict-safe writer transform evidence into candidate stories. The architecture clearly separates AI reasoning from changes to Jira. The enrichment and RL components can generate, evaluate, and improve a candidate story, but they cannot update Jira

Manuscript submitted to ACM

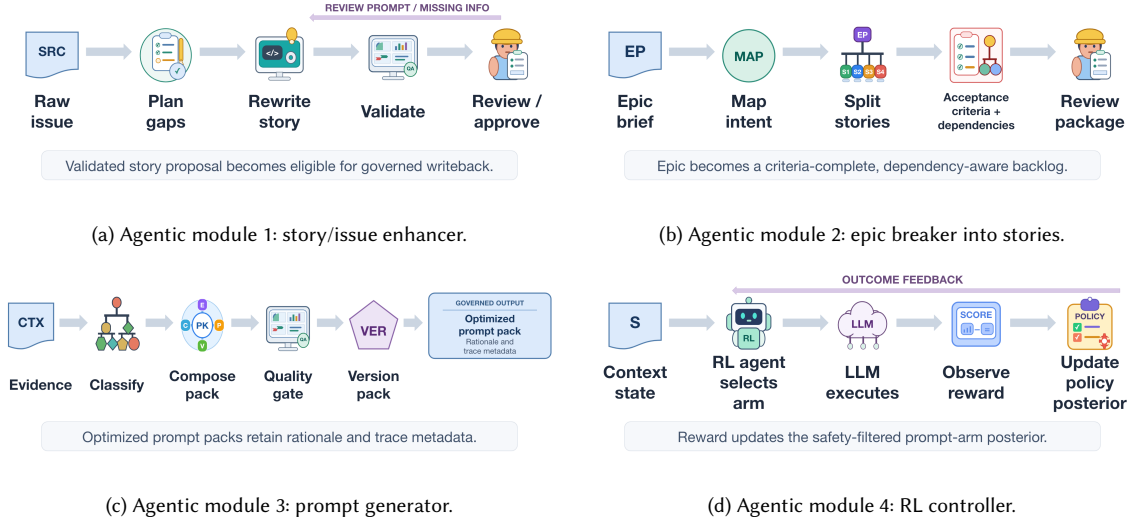


Fig. 3. Modular agent architecture: story/issue enhancement, epic-to-story decomposition, prompt-pack creation, and RL-based policy control. The panels expose agent boundaries hidden in the top-level architecture.

directly. A candidate can be written to Jira only after it passes validation and the confidence threshold, complies with the relevant policies, and receives human approval.

Figure 2: execution/control flow. Figure 2 shows the same components in execution order. A run acquires Jira context and retrieves requirement, design, code, and review signals. The orchestrator first creates a workflow record and selects an allowed prompt strategy. It then generates and scores candidate stories, applies policy checks, and records each step. Only approved candidates can be written to Jira. Each arrow in the figure represents a defined handoff between components, including input preparation, prompt-selection logging, draft scoring, policy recording, and writeback validation. Before updating Jira, the system checks for conflicting edits, safe retries (idempotency), and duplicate stories. If any check fails, the item remains in a review, refinement, conflict, or failure state, and Jira is not changed.

Figure 3: Internal agent modules. Figure 3 shows four internal agentic functions that are usually hidden inside a general “AI engine.” Agentic Module 1 examines an issue and identifies missing acceptance criteria, testability details, risks, and areas that need refinement. Agentic Module 2 divides a high-level epic into candidate stories and adds dependencies, acceptance criteria, and information for human review. Agentic Module 3 creates context-aware prompt packs by combining the issue text, reusable patterns, policy constraints, and identified information gaps. Agentic Module 4 acts as the RL controller. It selects an approved prompt strategy, called a policy arm, observes the result, and assigns a reward based on its quality. It then updates its posterior estimate, which represents how well each strategy is expected to perform in future tasks. This allows the controller to choose between defined strategies, such as conservative decomposition and balanced-context generation, while avoiding unsafe or unrestricted prompts.

The labels in Figures 1 and 2 denote executable contracts. Context and code retrieval are responsible for inputs normalization; *Human actions* describes reviewer intent; safe-arm selection and generation result in scored candidates; *Policy check* decides about confidence, role, validation, and approval; *Persist trace* records state, outcomes, and evidence

links. Approved creation reaches controlled writeback only after the source hash, idempotency, duplicate, and target checks. The safety boundary is hence determined by this rule rather than fluency of text.

3.1.2 Traceable Data Records. Every run captures scope, requester, time stamps, and idempotency information along with item state, confidence, readiness, policy flags, structured drafts, references to external evidence, decisions and notes on approval, and before/after writeback hashes. Since each piece of information is a first class record, the planning decision behind it may be subsequently reconstructed for empirical evaluation [31, 51], auditing [45], or incident analysis. Immutable event streams maintain event ordering, whereas mutable snapshots facilitate low-latency querying. Informational warnings alert about non-critical deficiencies, advisory warnings bring problems to reviewer attention, and blocking warnings stop mutation until the issue gets fixed.

3.2 Notation and Scale Conventions

Table 2 includes the notation used in the mathematical models and algorithms. Confidence and readiness vary in $[0, 1]$ unless otherwise specified. The notation $C_{10}(s) = 10C(s)$ denotes the corresponding value on the zero-to-ten scale of the interface. The creation gate is configured as $\tau_c = 0.9$, whereas offline replay employs the shaping threshold without writeback of $\tau_r = 0.8$. We use τ_e for the escalation. Subscripts s , t , k , and m stand for a story, refinement attempt, prompt arm, and evaluation method, respectively. In case of a base letter with engineering and statistical meanings, we qualify it with the appropriate subscript.

Table 2. Comprehensive notation used in the formal models, algorithms, operational equations, and evaluation.

Symbol	Definition
Planning, evidence, and constrained selection	
E, E_u	Epic under analysis; E_u is an epic unit supplied to an evaluation core.
s, s_i, s_j	Candidate story and indexed candidate stories.
S_{existing}	Stories already linked to E .
S_{cand}	Finite candidate-story set generated for E .
S_{new}	Candidate subset proposed for possible creation.
S_{eligible}	Duplicate-free candidates satisfying confidence, approval, validation, and policy-allow conditions.
S_{create}	Stories successfully added to the controlled creation set.
S^*	Augmented plan $S_{\text{existing}} \cup S_{\text{new}}$.
$\mathcal{D}, \mathcal{D}_{\text{ext}}, \mathcal{D}_{\text{jira}}$	Complete evidence corpus, externally retrieved evidence, and Jira-embedded evidence.
$\mathbf{m}(s)$	Candidate metric tuple containing confidence, readiness, utility, ambiguity, risk, approval, and source hash.
$C(s), C_s$	Normalized confidence in $[0, 1]$ for story s and its algorithmic shorthand.
$C_{10}(s), C_{10,t}$	Displayed confidence for story s and at refinement attempt t on the zero-to-ten UI scale, with $C_{10}(s) = 10C(s)$.
$R(s), R_s, R_s(\mathbf{x})$	Readiness of story s , its algorithmic shorthand, and readiness viewed as a function of evidence coverage.
$U(s), \text{Amb}(s), \rho(s)$	Expected utility, ambiguity, and risk of candidate s .
$A(s), A_s$	Explicit human-approval indicator and its algorithmic shorthand; both are binary.
$V(s), V_s$	Validator result for schema, contradiction, and required-field checks; both are binary.
$\Pi, \Pi(s), g_{\Pi}(s)$	Versioned governance policy, its decision for s , and the binary policy-allow indicator.
$I_{\text{ctx}}(s)$	Binary indicator that story s is operating with limited external context.
$I_{\text{esc}}(s), I_{\text{create}}(s)$	Binary escalation-required and creation-eligibility predicates for story s .
$h_s, H(\cdot)$	Source-state hash stored at decision time and deterministic cryptographic hash function.
$\tau_c, \tau_{10}, \tau_e, \tau_r$	Normalized creation threshold (default 0.9), its displayed equivalent $\tau_{10} = 10\tau_c$, escalation threshold, and offline replay shaping threshold (0.8).
y_s	Binary selection variable; $y_s = 1$ selects s for the augmented plan.
$\mathbb{K}[\cdot]$	Indicator function, equal to one when its predicate is true and zero otherwise.
$\text{summary}(s), \text{norm}(\cdot)$	Story summary and deterministic lowercase/alphanumeric normalization.

Continued on next page

Table 2 (continued)

Symbol	Definition
$\sim_n, \mathcal{F}_{\text{dup}}$	Normalized-summary equivalence relation and duplicate-free candidate domain.
$\lambda_\rho, \lambda_{\text{Amb}}$	Non-negative objective weights for risk and ambiguity.
$\text{SP}(s), B_{\text{sprint}}$	Story-point estimate for candidate s and sprint story-point budget.
$\text{dep}(s_i, s_j), \text{team}(s), g, \text{cap}_g, \text{ord}(s)$	Dependency indicator, assigned team, team index, team-capacity bound, and release-order position.
$f(\cdot), \xi_{\text{scope}}, \xi_{\text{integration}}, \xi_{\text{risk}}, \xi_{\text{unknown}}$	Story-point mapping and normalized complexity priors were used for the initial assignment.
$\mathcal{W}_{\text{create}}$	Result of the controlled generation operation on S_{eligible} .
Readiness, confidence, and calibration	
$\mathbf{x} = (x_d, x_a, x_p, x_e, x_h, x_g)$	Binary vector of evidence for the description, acceptance criteria, dependencies, external evidence, activity history, and edge cases.
$R_{\text{src}}(\mathbf{x})$	Contribution of the ready candidate, represented as a weighted sum of the evidence coverage vector $\mathbf{x}$.
$B_{\text{gen}}, P_{\text{gap}}$	Bonus and penalty for the generated content depending on whether there is evidence or evidence contradiction.
z_i, w_i, b_{max}	The quality facet of the generated content and the weight of the facet; the bounded generated bonus total.
$\eta_d, \eta_a, \eta_p, \eta_e$	Gap-penalty weights (non-negative) for missing description, acceptance criteria, dependencies, and external evidence.
$\delta_C, \gamma_{\text{ceiling}}$	Bounded confidence update and the evidence-based confidence ceiling.
$\text{clip}(u, a, b)$	Clipping operator $\min(\max(u, a), b)$.
ECE, Brier	Expected calibration error and Brier score; both require post-delivery labels.
Prompt policy, reward, and convergence	
p_k, k, K_{arm}	Prompt pack associated with the arm k , the arm index, and the number of prompt-policy arms.
$\theta_k, \hat{\theta}_{k,t}$	Success probability of the underlying arm and its Thompson sample at attempt t .
α_k, β_k	Soft counts of successes and failures for the Beta posterior of arm k .
$n_k, \bar{r}_k$	Number of recorded outcomes and running mean reward for arm k .
$\mathbf{h}_t$	Context feature state used for choosing the policy at attempt t .
$\mathcal{K}, \mathcal{K}_{\text{safe}}(\mathbf{h}_t)$	Complete set of prompt arms and the subset that satisfies every governance condition at attempt t .
$\chi_j(k, \mathbf{h}_t)$	Binary governance check for arm k under condition j and context $\mathbf{h}_t$; 1 means satisfied and 0 means rejected.
j, n_χ	Governance-condition index and total number of governance conditions.
$C_{10,t}^{\text{before}}, C_{10,t}^{\text{after}}$	Displayed confidence before and after the recorded outcome for selected arm k_t .
d_t, w_t	Recorded reviewer decision and writeback status for the selected arm.
r_t, q_t	Signed reward clipped to $[-1, 1]$ and its fractional Beta success credit $q_t = (r_t + 1)/2$.
T, t, k_t	Horizon of attempts, attempt number, and arm chosen at attempt t .
$\ell_t, \ell(t)$	Confidence gap at attempt t and its continuous interpolation.
$\kappa, \hat{\kappa}_t$	Effective lag-convergence rate and its attempt-level estimate.
$d_t(t), d_{\text{max}}$	Lag disturbance and its assumed absolute bound.
$\mathbb{E}[\cdot], \Pr(\cdot), \arg \max$	Expectation, probability, and maximizing-argument operators.
$\mathcal{L}\{\cdot\}, s_L, \Lambda_\ell(s_L), D_\ell(s_L)$	Laplace-transform operator, Laplace-domain variable, transform of $\ell(t)$, and transform of $d_t(t)$.
ϵ	Positive numerical guard for rate estimation.
$\mathcal{X}, \mathcal{P}, C, \mathcal{T}_{\text{audit}}$	Connector set, prompt-policy state, retrieved context pack, and durable attempt/audit trace.
Approval-bound mutation and batch outcomes	
$\mathcal{E}_a, \mathcal{E}_t$	Approved source-bound transaction envelope and the corresponding runtime envelope evaluated immediately before mutation.
$r_{\text{run}}, i_{\text{issue}}$	Run and issue identifiers bound into $\mathcal{E}_a$.
v_s, v_d, h_d	Approved source revision, draft version, and draft payload hash; the approved source hash is h_s .
v_Π, h_Π	Approved governance-policy version and policy-snapshot hash.
$\mathcal{T}_{\text{target}}$	Canonical set of allowed mutation targets.
$a_{\text{dec}}, u_{\text{rev}}, t_a$	Approval decision, reviewer identifier, and approval time.
I_{tx}	Transaction identifier $H(\mathcal{E}_a)$.
K_{A2}, K_{A3}	A2 payload-level idempotency key and A3 source-bound envelope key.
A0, A1, A2, A3	Mutation-safety arrangements: ordinary approval/logging, source-hash conditional write, source hash with atomic idempotency, and the source-bound transaction envelope.
$h_{\text{current}}, h_{\text{expected}}$	Runtime source hash and approved source hash expected by writeback, where $h_{\text{expected}} := h_s$.

Continued on next page

Table 2 (continued)

Symbol	Definition
$\text{permit}(\mathcal{E}_a, \mathcal{E}_t)$	Binary mutation authorization predicate.
$\text{write}(s)$	Binary write indicator; it equals one when story s is written to Jira and zero otherwise.
$o_s, \mathcal{B}_{\text{batch}}, \text{status}(\mathcal{B}_{\text{batch}})$	Per-story creation outcome, candidate batch, and precedence-derived batch status.
Evaluation and statistical analysis	
$B0, B1, B2, B3, m$	Manual reference, template/rule core, one-shot drafting control, Jira Enhancer, and method index.
$Q_m, \text{Eff}_m, G_m, \text{Composite}_m, \text{Time}_m$	Quality, efficiency, governance, composite, and time indices for method m .
w_Q, w_{Eff}, w_G	Fixed non-negative composite-index weights that sum to one.
$\Delta_{B3,m}, \Delta$	Mean composite difference between B3 and baseline method $m \in \{B0, B1, B2\}$, and a generic resampled mean difference.
H_0, H_1, μ_m	Null/alternative hypotheses and mean composite for method m .
Q_{omni}	Matched-study permutation omnibus statistic.
$H_{\text{KW}}, F_{\text{ANOVA}}, U_{\text{MW}}$	Kruskal-Wallis, one-way ANOVA, and Mann-Whitney statistics.
$r_{rb}, \delta_{\text{Cliff}}$	Paired rank-biserial and Cliff's effect sizes.
$X, Y, x_i, y_j, n_x, n_y, \bar{X}, \bar{Y}$	Two comparison samples, indexed observations, sample sizes, and sample means.
$n, \text{SD}, \text{IQR}, \text{P90}$	Generic sample size, sample standard deviation, interquartile range, and 90th percentile.
$p, p_{\text{Holm}}, \alpha_{\text{sig}}, M_{\text{cmp}}$	Raw or reported probability value, Holm-adjusted probability value, familywise significance level, and number of pairwise comparisons.
$\text{CI}, \text{ICC}(2, 1)$	Confidence interval and two-way random-effects absolute-agreement intraclass correlation.
service_mode	Connector operating mode: full-context or reduced-context.

3.3 Formal Problem Formulation

Let E be an epic, S_{existing} the set of already-linked stories, $\mathcal{D}$ the available evidence corpus, and Π a versioned governance policy. The evidence corpus is decomposed as $\mathcal{D} = \mathcal{D}_{\text{ext}} \cup \mathcal{D}_{\text{jira}}$, where $\mathcal{D}_{\text{ext}}$ contains external evidence from Confluence/Bitbucket and $\mathcal{D}_{\text{jira}}$ contains embedded design context in the Jira description. This distinction is important because, when external links are unavailable but the epic description contains useful design rules, APIs, guardrails, or tests, the system can still produce evidence-grounded candidates while identifying their provenance as Jira-embedded context. The system constructs a finite candidate set S_{cand} and selects a subset $S_{\text{new}} \subseteq S_{\text{cand}}$ for possible creation. Here, requirements are modeled as evolving and negotiated objects rather than fixed text [46]. This agrees with established RE views that requirements are incomplete and shaped by stakeholders [44]. It also follows requirements-quality guidance on clarity, verifiability, feasibility, and traceability [25]. Each candidate s is represented by the tuple

$$\mathbf{m}(s) = (C(s), R(s), U(s), \text{Amb}(s), \rho(s), A(s), h_s), \quad (1)$$

where $C(s)$ is normalized confidence, $R(s)$ is readiness, $U(s)$ is expected utility, $\text{Amb}(s)$ is ambiguity, $\rho(s)$ is risk, $A(s) \in \{0, 1\}$ is human approval, and h_s is the source-state hash at the time of decision.

Normalized story identity is modeled through an equivalence relation $\sim_n$:

$$s_i \sim_n s_j \iff \text{norm}(\text{summary}(s_i)) = \text{norm}(\text{summary}(s_j)), \quad (2)$$

Norm operates by lowercasing the text and removing non-alphanumeric separators from it. The duplicate-free candidate domain is thus:

$$\mathcal{F}_{\text{dup}} = \{s \in S_{\text{cand}} \mid \nexists s' \in S_{\text{existing}} : s \sim_n s'\}. \quad (3)$$

This formulation ensures that generation of work items does not result in duplicate tasks due to normalization that would turn different cases of task summaries into the same summary.

The basic feasibility relation is:

$$S^* = S_{\text{existing}} \cup S_{\text{new}}, \quad (4)$$

$$\forall s \in S_{\text{new}} : C(s) \geq \tau_c, \quad (5)$$

$$\forall s \in S_{\text{new}} : A(s) = 1 \text{ (approved)}, \quad (6)$$

$$S_{\text{eligible}} = \{s \in S_{\text{new}} \cap \mathcal{F}_{\text{dup}} \mid C(s) \geq \tau_c \wedge A(s) = 1 \wedge V(s) = 1 \wedge \Pi(s) = \text{ALLOW}\}, \quad (7)$$

$$\mathcal{W}_{\text{create}} = \text{Create}(S_{\text{eligible}}), \quad (8)$$

where $\tau_c \in [0, 1]$ is the normalized creation threshold (default $\tau_c = 0.9$, displayed as $C_{10}(s) = 9$, i.e., 9/10), and $V(s) \in \{0, 1\}$ is the validation indicator for candidate story s . It equals one only when the candidate passes the schema, contradiction, and required-field checks. The condition $\Pi(s) = \text{ALLOW}$ means that the active governance policy permits the candidate to be created.

This formulation embodies one of the key principles: high quality of generation alone does not make mutation possible. Creation is an act requiring machine confidence as well as human authority, so recommendation can never transform into an unauthorized action.

The feasibility rules above determine which candidate stories are allowed to reach the creation stage. However, several candidates may satisfy these rules, and the system must still decide which ones should be included in the augmented plan.

For each candidate $s \in S_{\text{cand}}$, we define a binary decision variable $y_s \in \{0, 1\}$. The value $y_s = 1$ means that the candidate is selected, while $y_s = 0$ means that it is not selected. The system uses the following objective:

$$\max_{\{y_s\}} \sum_{s \in S_{\text{cand}}} y_s \left(U(s) - \lambda_\rho \rho(s) - \lambda_{\text{Amb}} \text{Amb}(s) \right), \quad (9)$$

subject to:

$$y_s \leq \mathbb{I}[C(s) \geq \tau_c], \quad (10)$$

$$y_s \leq A(s), \quad (11)$$

$$y_s \leq V(s), \quad (12)$$

$$y_s \leq \mathbb{I}[\Pi(s) = \text{ALLOW}], \quad (13)$$

$$y_s \leq 1 - \mathbb{I}[\exists s' \in S_{\text{existing}} : \text{norm}(\text{summary}(s)) = \text{norm}(\text{summary}(s'))], \quad (14)$$

$$y_s \in \{0, 1\}. \quad (15)$$

The objective gives preference to candidates with high expected utility $U(s)$. It reduces the selection score when the candidate has high risk $\rho(s)$ or high ambiguity $\text{Amb}(s)$. The non-negative weights λ_ρ and λ_{Amb} determine how strongly risk and ambiguity affect the selection.

The constraints ensure that a candidate can be selected only when its confidence reaches τ_c , it has human approval, it passes validation, and the active policy allows its creation. The duplicate constraint prevents the selection of a candidate whose normalized summary matches an existing story. Here, $\mathbb{I}[\cdot]$ equals one when the stated condition is true and zero otherwise, while $\text{norm}(\cdot)$ performs deterministic summary normalization. Therefore, expected utility helps the system rank suitable candidates, but it cannot override confidence, approval, validation, policy, or duplicate-safety requirements.

In scheduling applications, the objective can be extended with portfolio constraints:

$$\sum_{s \in S_{\text{cand}}} y_s \text{SP}(s) \leq B_{\text{sprint}}, \quad (16)$$

$$\sum_{s \in S_{\text{cand}}} y_s \mathbb{I}[\text{team}(s) = g] \leq \text{cap}_g, \quad (17)$$

$$\text{dep}(s_i, s_j) = 1 \Rightarrow \text{ord}(s_i) < \text{ord}(s_j), \quad (18)$$

where $\text{SP}(s)$ is the story-point estimate for candidate s , B_{sprint} is the sprint budget, cap_g is the capacity bound for team g , and $\text{ord}(\cdot)$ is a release-ordering function. This demonstrates that the proposed mechanism can support both decomposition quality and downstream planning feasibility.

Two invariants follow directly from the constraints:

$$y_s = 1 \Rightarrow C(s) \geq \tau_c, \quad (19)$$

$$y_s = 1 \Rightarrow A(s) = 1 \wedge V(s) = 1 \wedge \Pi(s) = \text{ALLOW}. \quad (20)$$

Consequently, selected stories are both confidence-feasible and governance-feasible. These invariants support the multi-gate creation rule used later in the policy layer.

Proposition 1: eligibility soundness. For any candidate $s \in S_{\text{cand}}$, if $y_s = 1$ in the constrained optimization problem above, then s is confidence-feasible, approval-feasible, validation-feasible, policy-feasible, and duplicate-free:

$$y_s = 1 \Rightarrow C(s) \geq \tau_c \wedge A(s) = 1 \wedge V(s) = 1 \wedge \Pi(s) = \text{ALLOW} \wedge s \in \mathcal{F}_{\text{dup}}. \quad (21)$$

Proof sketch. The implication follows by substituting $y_s = 1$ into all five eligibility upper-bound constraints. Because y_s is binary, every upper bound must evaluate to one; otherwise, the corresponding constraint would be violated. Hence, the confidence, approval, validation, policy, and duplicate constraints hold simultaneously.

Proposition 2: Mutation Requires Approval. If $A(s) = 0$, then s cannot be selected for creation under the optimization model:

$$A(s) = 0 \Rightarrow y_s = 0. \quad (22)$$

Proof sketch. Approval condition says $y_s \leq A(s)$. Given $A(s) = 0$ and $y_s \in \{0, 1\}$, the only feasible value is $y_s = 0$. It establishes the human-authority invariant.

The system is set up for use in cases where confidence and approval will be the main factors involved, rather than label associated with the generated output.

3.4 Readiness and Confidence Model

The system calculates readiness and confidence in four steps. It first records which evidence is available. It then calculates a base evidence score, adds a bounded contribution from generated content, and subtracts penalties for missing evidence. Finally, it converts the resulting readiness score into a confidence value.

Let $\mathbf{x} \in \{0, 1\}^6$ represent the available evidence:

$$x_d = \mathbb{I}[\text{description exists}], \quad (23)$$

$$x_a = \mathbb{I}[\text{acceptance criteria exist}], \quad (24)$$

$$x_p = \mathbb{I}[\text{dependencies exist}], \quad (25)$$

$$x_e = \mathbb{I}[\text{external evidence available}], \quad (26)$$

$$x_h = \mathbb{I}[\text{activity history available}], \quad (27)$$

$$x_g = \mathbb{I}[\text{edge cases available}]. \quad (28)$$

Each indicator equals one when the corresponding evidence is available and zero when it is missing.

The system uses these indicators to calculate the base evidence score:

$$R_{\text{src}}(\mathbf{x}) = 0.25x_d + 0.25x_a + 0.15x_p + 0.15x_e + 0.10x_h + 0.10x_g. \quad (29)$$

The weights are non-negative and add up to one. Therefore, $R_{\text{src}}(\mathbf{x})$ remains within $[0, 1]$. Description and acceptance criteria receive the largest weights because they provide the main information needed to review a candidate story.

In create-run mode, the system may add a bounded bonus for useful generated content:

$$B_{\text{gen}} = \sum_i w_i z_i, \quad (30)$$

where $z_i \in [0, 1]$ represents a generated-text quality facet, such as the problem statement, acceptance details, risk model, or non-functional requirements. The weight of each facet satisfies $w_i \geq 0$, and the total weight satisfies $\sum_i w_i \leq b_{\text{max}}$. This upper bound prevents generated text from receiving too much credit when supporting evidence is missing.

The system also applies a penalty for important evidence gaps:

$$P_{\text{gap}} = \eta_d(1 - x_d) + \eta_a(1 - x_a) + \eta_p(1 - x_p) + \eta_e(1 - x_e) + \dots \quad (31)$$

The non-negative parameters η_d , η_a , η_p , and η_e control the penalty for each type of missing evidence. The remaining terms follow the same pattern for any additional evidence gaps.

The system combines the base score, generated-content bonus, and gap penalty to calculate readiness:

$$R(s) = \text{clip}(R_{\text{src}}(\mathbf{x}) + B_{\text{gen}} - P_{\text{gap}}, 0, 0.99), \quad (32)$$

$$C(s) = \text{clip}(R(s) + \delta_C, 0, \gamma_{\text{ceiling}}). \quad (33)$$

Here, $\text{clip}(u, a, b) = \min(\max(u, a), b)$ keeps a value between the lower bound a and the upper bound b . The term δ_C is a bounded confidence adjustment. The interface displays confidence as $C_{10}(s) = 10C(s)$ and rounds the result for easier reading. The evidence-based limit $\gamma_{\text{ceiling}} \leq 1$ prevents the system from assigning very high confidence when critical evidence is missing, even if the generated draft appears coherent.

To show how evidence affects readiness, we may also write readiness as $R_s(\mathbf{x})$. When the generated-content bonus and gap penalty are held fixed, adding evidence cannot reduce the readiness score. Therefore,

$$\mathbf{x}' \geq \mathbf{x} \implies R_s(\mathbf{x}') \geq R_s(\mathbf{x}), \quad (34)$$

where $\mathbf{x}' \geq \mathbf{x}$ means that every evidence indicator in $\mathbf{x}'$ is equal to or greater than the corresponding indicator in $\mathbf{x}$.

Lemma 1: confidence is bounded. For any generated candidate s , $R(s) \in [0, 0.99]$ and $C(s) \in [0, \gamma_{\text{ceiling}}]$. *Proof sketch.* Both values are calculated using a clipping operator. By definition, $\text{clip}(u, a, b)$ belongs to $[a, b]$. Thus, $R(s)$ lies in $[0, 0.99]$ and $C(s)$ lies in $[0, \gamma_{\text{ceiling}}]$, irrespective of the raw evidence score, generated bonus, or gap penalty.

Lemma 2: monotonicity under fixed penalties. For $\mathbf{x}', \mathbf{x} \in \{0, 1\}^6$ such that $\mathbf{x}' \geq \mathbf{x}$ componentwise, $R_s(\mathbf{x}') \geq R_s(\mathbf{x})$ if $B_{\text{gen}}, P_{\text{gap}}$, and γ_{ceiling} are fixed. *Proof sketch.* The evidence score is calculated as a non-negative weighted sum over $\mathbf{x}$, with all weights being non-negative. Hence, $R_{\text{src}}(\mathbf{x}') \geq R_{\text{src}}(\mathbf{x})$. Applying the same generated bonus and penalty, followed by a monotone clipping function, maintains this inequality.

In practice, the blended formalism shows reviewers how the confidence score was calculated. This allows them to understand whether a particular score is low because of missing acceptance criteria, insufficient evidence, or complex dependencies.

Calibration scope. In this study, confidence is a bounded score based on the available evidence. It is not treated as a direct probability of successful delivery. We examined how the score changes during refinement and use it together with validation, policy checks, and human approval. Measuring calibration requires later outcomes, such as story carryover, defects, or reopened stories. These outcomes were not available in the two deployment waves. When they become available, ECE and the Brier score can be used to measure how closely confidence matches the actual delivery results [11, 20].

Additionally, the model accounts for context-dependent penalties. For example, unresolved architectural dependencies add an uncertainty term to P_{gap} even when the corresponding fields are populated. This approach helps avoid high confidence for tickets that appear complete but remain semantically weak, a concern reported in agile requirements, requirements-smell, and issue-report quality research [10, 18, 41].

3.5 RL Prompt Selection Model

The adaptive component in use is a contextual multi-armed bandit [35]. This can be viewed as a restricted short-horizon reinforcement-learning setting, but it does not model a long-horizon Markov decision process or estimate a value function over the workflow state space. We nevertheless refer to it as the *RL layer* because that term is used in the implementation and architecture, while its learning scope remains limited.

3.5.1 Bandit Definition. Let each prompt pack p_k be an arm in a stochastic multi-armed bandit [5]. Posterior per arm is:

$$\theta_k \sim \text{Beta}(\alpha_k, \beta_k). \quad (35)$$

At each attempt t [52]:

- (1) sample $\tilde{\theta}_{k,t}$ from the posterior of each arm in $\mathcal{K}_{\text{safe}}(\mathbf{h}_t)$,
- (2) choose $k_t = \arg \max_{k \in \mathcal{K}_{\text{safe}}(\mathbf{h}_t)} \tilde{\theta}_{k,t}$,
- (3) apply selected prompt and observe reward proxy,
- (4) update $(\alpha_{k_t}, \beta_{k_t})$.

The posterior-update procedure is shown in Algorithm 2.

Eventually, the system will be able to learn which types of prompts work well for different issue types. For example, technical-debt issues may require shorter implementation prompts, while user-facing issues may require more elaborate prompts to communicate the requirements.

Expected performance of arm k :

$$\mathbb{E}[\theta_k] = \frac{\alpha_k}{\alpha_k + \beta_k}. \quad (36)$$

3.5.2 Policy Arms in the RL Controller. The RL layer operates over a small number of prompt-policy *arms*. An arm is not merely a prompt template; it is a bounded strategy that defines decomposition style, evidence use, validation strictness, and acceptable risk. Separating the arms makes the learning behavior easier to audit because each policy has a defined purpose and risk envelope.

Table 3. Prompt-policy arms for the RL controller. Arms A–F are illustrative examples used to explain the arm-selection process.

Arm	Strategy	Intended use
A	Conservative decomposition, $\bar{r}_A = 0.61$	Prefer minimal, low-risk story creation when evidence is sparse or governance risk is high.
B	Balanced context strategy, $\bar{r}_B = 0.74$	Balance coverage, acceptance detail, and reviewer burden when evidence quality is sufficient.
C	Aggressive rewrite, $\bar{r}_C = 0.58$	Explore broader restructuring only when the epic is highly ambiguous and reviewer intervention is expected.
D	Acceptance-criteria enrichment, $\bar{r}_D = 0.5340$	Produce concise, testable acceptance criteria tied to the Jira summary and available evidence.
E	Dependency-aware planning, $\bar{r}_E = 0.4667$	Prioritize dependency sequencing, blockers, prerequisites, integration constraints, and validation checkpoints.
F	Evidence-context enrichment, $\bar{r}_F = 0.4875$	Ask targeted reviewer questions, keep the scope narrow, and record unknowns when external evidence is sparse.

For each context state $\mathbf{h}_t$, the system first applies a safety filter. The filter removes any prompt arm that does not meet the governance conditions, such as sufficient evidence, an allowed workflow state, and a valid write target. The remaining safe arms are defined as:

$$\mathcal{K}_{\text{safe}}(\mathbf{h}_t) = \{k \in \mathcal{K} \mid \chi_j(k, \mathbf{h}_t) = 1 \text{ for all } j = 1, \dots, n_\chi\}. \quad (37)$$

Here, $\mathcal{K}$ is the complete set of prompt arms, and $\mathcal{K}_{\text{safe}}(\mathbf{h}_t)$ is the subset allowed at attempt t . The context state $\mathbf{h}_t$ contains the information used to make the decision. The function $\chi_j(k, \mathbf{h}_t) \in \{0, 1\}$ checks whether arm k satisfies governance condition j in this context. A value of 1 means that the condition is satisfied, whereas 0 means that the arm is rejected. The index j identifies a governance condition, and n_χ is the total number of conditions. Thus, an arm enters the safe set only when every condition returns 1.

After this filtering step, the controller applies Thompson sampling only to the safe arms. At attempt t , it draws a value from the Beta posterior of each safe arm and selects the arm with the largest sampled value:

$$\begin{aligned} \tilde{\theta}_{k,t} &\sim \text{Beta}(\alpha_k, \beta_k), & k \in \mathcal{K}_{\text{safe}}(\mathbf{h}_t), \\ k_t &= \arg \max_{k \in \mathcal{K}_{\text{safe}}(\mathbf{h}_t)} \tilde{\theta}_{k,t}. \end{aligned} \quad (38)$$

Therefore, the controller can explore different prompt strategies, but it cannot select an arm that fails a governance condition.

3.5.3 Confidence-Guided Refinement. For each refinement attempt t , the system records the displayed confidence $C_{10,t} \in [0, 10]$. The remaining gap between this confidence and the target τ_{10} is calculated as:

$$\ell_t = \max(0, \tau_{10} - C_{10,t}), \quad (39)$$

where ℓ_t represents the confidence gap. If the target is not reached, the system uses the latest gap to guide the next refinement attempt. It also stores the complete confidence trajectory for auditing and debugging. The refinement process ends when the confidence reaches the target or when the maximum number of retries is reached. The complete refinement and writeback workflow is shown in Algorithm 1.

3.5.4 Laplace-Domain Convergence View. As the implemented refinement loop operates in a discrete time fashion, its runtime state is encoded in $\{\ell_t\}_{t=1}^T$. In order to examine its convergence properties we interpolate the lag trajectory in the form of $\ell(t)$ and analyze it in the Laplace domain. Such interpolation serves as a diagnostic tool, rather than a substitute for the discrete algorithm, and it is used as a cheap way to verify whether confidence lag decays during iterations.

Let the idealized lag dynamics be:

$$\frac{d\ell(t)}{dt} = -\kappa\ell(t) + d_\ell(t), \quad (40)$$

where $\kappa > 0$ represents the convergence rate of the chosen prompt policy and $d_\ell(t)$ denotes a bounded disturbance from lack of evidence, conflicting context, or reviewer feedback. Let s_L denote the Laplace-domain variable, $\Lambda_\ell(s_L) = \mathcal{L}\{\ell(t)\}$, and $D_\ell(s_L) = \mathcal{L}\{d_\ell(t)\}$. The transformed dynamics are:

$$s_L \Lambda_\ell(s_L) - \ell(0) = -\kappa \Lambda_\ell(s_L) + D_\ell(s_L), \quad (41)$$

and therefore:

$$\Lambda_\ell(s_L) = \frac{\ell(0) + D_\ell(s_L)}{s_L + \kappa}. \quad (42)$$

In the no-disturbance case, $D_\ell(s_L) = 0$, so:

$$\Lambda_\ell(s_L) = \frac{\ell(0)}{s_L + \kappa}. \quad (43)$$

The only pole is at $s_L = -\kappa$. Hence the lag dynamics are stable when $\kappa > 0$. By the final value theorem,

$$\lim_{t \rightarrow \infty} \ell(t) = \lim_{s_L \rightarrow 0} s_L \Lambda_\ell(s_L) = 0, \quad (44)$$

This implies that confidence lag converges to zero when the refinement policy has positive effective gain and no persistent disturbance. If $|d_\ell(t)| \leq d_{\max}$, then the steady-state lag magnitude is bounded above by $d_{\max}/\kappa$; hence, larger κ implies faster and tighter convergence.

In implementation, κ can be estimated from observed attempts as follows:

$$\widehat{\kappa}_t = \max\left(0, \log \frac{\ell_{t-1} + \epsilon}{\ell_t + \epsilon}\right), \quad (45)$$

where $\epsilon > 0$ avoids division by zero. This estimate is logged alongside the confidence trajectory and can help determine whether another prompt attempt is likely to help.

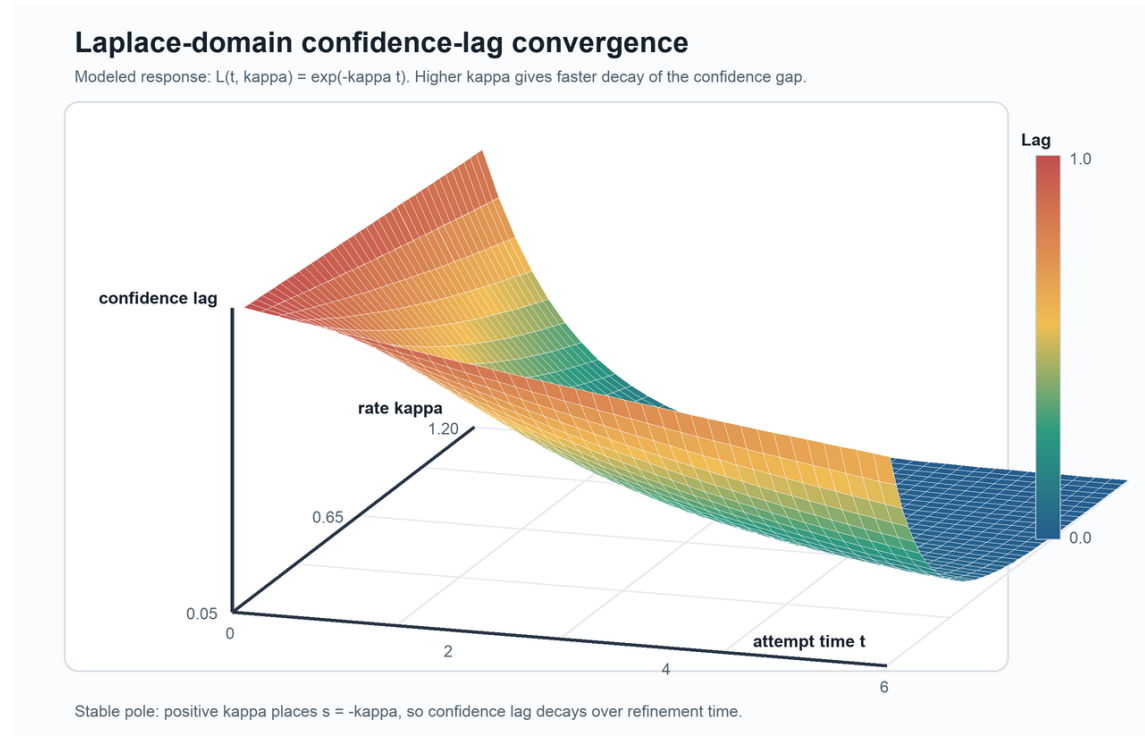


Fig. 4. Laplace-inspired confidence-lag convergence surface for normalized lag $\ell(t; \kappa) = e^{-\kappa t}$. Larger κ values yield faster lag decay, consistent with the pole-location condition $s_L = -\kappa < 0$.

3.5.5 *Algorithmic Realization.* Algorithms 1 and 2 show how the model works in practice.

Algorithm 1 presents the complete workflow. It retrieves context and generates candidate stories. It selects a safe prompt arm and refines each story. A story reaches Jira only after validation, policy checks, and human approval.

Algorithm 2 presents the prompt-policy update. It computes a signed reward from the confidence change, reviewer decision, and writeback status. The bounded reward is mapped to fractional success credit and updates only the selected arm's Beta posterior. Thompson sampling uses the updated posterior during future prompt-arm selection.

Algorithm 1: End-to-End RL-Governed Epic-to-Story Inference Pipeline

Input: Epic E , connectors $\mathcal{X}$ (Jira/Confluence/Bitbucket), policy state $\mathcal{P}$, thresholds (τ_c, τ_e) , max attempts T

Output: Created story set S_{create} , audit trace $\mathcal{T}_{\text{audit}}$

```

1  $\tau_{10} \leftarrow 10\tau_c$ ;  $C \leftarrow \text{RetrieveContext}(E, \mathcal{X})$ ;  $\mathcal{T}_{\text{audit}} \leftarrow \emptyset$ 
2  $S_{\text{cand}} \leftarrow \text{GenerateCandidates}(E, C)$ 
3 foreach  $s \in S_{\text{cand}}$  do
4    $t \leftarrow 1$ ;  $\ell_0 \leftarrow \tau_{10}$ ;  $A_s \leftarrow 0$ 
5   while  $t \leq T$  do
6      $k_t \leftarrow \text{SelectPromptArm}(\mathcal{P}, s, C)$ 
7      $s \leftarrow \text{EnrichWithPrompt}(s, k_t, C)$ 
8      $(R_s, C_s) \leftarrow \text{ScoreReadinessConfidence}(s, C)$ 
9      $C_{10,t} \leftarrow 10C_s$ ;  $\ell_t \leftarrow \max(0, \tau_{10} - C_{10,t})$ 
10     $\hat{\kappa}_t \leftarrow \text{EstimateLaplaceRate}(\ell_{t-1}, \ell_t)$ 
11     $V_s \leftarrow \text{ConsistencyValidate}(s)$ 
12     $\text{LogAttempt}(\mathcal{T}_{\text{audit}}, E, s, t, k_t, R_s, C_s, \ell_t, \hat{\kappa}_t, V_s)$ 
13    if  $(C_s \geq \tau_c)$  and  $(V_s = 1)$  then
14      break
15     $t \leftarrow t + 1$ 
16   $I_{\text{esc}}(s) \leftarrow \mathbb{I}[C_s < \tau_e] \vee I_{\text{ctx}}(s)$ 
17   $(g_{\Pi}(s), A_s) \leftarrow \text{PolicyAndHumanGate}(s, I_{\text{esc}}(s))$ 
18   $I_{\text{create}}(s) \leftarrow \mathbb{I}[C_s \geq \tau_c \wedge A_s = 1 \wedge V_s = 1 \wedge g_{\Pi}(s) = 1 \wedge s \in \mathcal{F}_{\text{dup}}]$ 
19  if  $I_{\text{create}}(s) = 1$  then
20    if  $\text{HashCheckAndIdempotencyPass}(s)$  then
21       $\text{Writeback}(s)$ ;  $\text{AddToCreateSet}(S_{\text{create}}, s)$ 
22 return  $(S_{\text{create}}, \mathcal{T}_{\text{audit}})$ 

```

Algorithm 2: RL Prompt Policy Update (Bandit + Governance-Aware Reward)**Input:** Selected-arm state $(\alpha_{k_t}, \beta_{k_t}, n_{k_t}, \bar{r}_{k_t})$ and recorded outcome $(C_{10,t}^{\text{before}}, C_{10,t}^{\text{after}}, d_t, w_t)$ **Output:** Updated selected-arm state $(\alpha_{k_t}, \beta_{k_t}, n_{k_t}, \bar{r}_{k_t})$

```

1  $r_t \leftarrow (C_{10,t}^{\text{after}} - C_{10,t}^{\text{before}}) / 10$ 
2  $r_t \leftarrow r_t + 0.2\mathbb{I}[d_t = \text{approve}] - 0.2\mathbb{I}[d_t \in \{\text{reject}, \text{regenerate}\}]$ 
3  $r_t \leftarrow r_t + 0.1\mathbb{I}[w_t = \text{written}] - 0.1\mathbb{I}[w_t = \text{conflict}]$ 
4  $r_t \leftarrow \text{clip}(r_t, -1, 1)$ 
5  $q_t \leftarrow (r_t + 1) / 2$ 
6  $(\alpha_{k_t}, \beta_{k_t}) \leftarrow (\alpha_{k_t} + q_t, \beta_{k_t} + 1 - q_t)$ 
7  $n_{k_t} \leftarrow n_{k_t} + 1$ 
8  $\bar{r}_{k_t} \leftarrow \bar{r}_{k_t} + (r_t - \bar{r}_{k_t}) / n_{k_t}$ 
9 return  $(\alpha_{k_t}, \beta_{k_t}, n_{k_t}, \bar{r}_{k_t})$ 

```

3.6 Epic-to-Story Decomposition Method*3.6.1 Candidate Generation.* Using epic context and existing stories, the system generates missing capability archetypes:

- UI/interaction story,
- API/contract story,
- persistence/writeback story,
- validation/regression story.

The method suppresses archetypes already covered by existing summaries using lexical and semantic overlap tests.

This prevents “story inflation”, a common failure mode in which assistants generate redundant tickets that differ in wording but represent the same underlying work.

The current implementation also treats substantive embedded epic descriptions as first-class evidence. When a Jira epic contains design signals such as solution-design headings, scaling rules, formulas, API/workflow descriptions, validation requirements, rollback guardrails, or test obligations, the resolver emits a `jira_embedded_context` evidence record. This makes decomposition robust to a common enterprise scenario: Confluence links may be inaccessible during execution, yet the epic may still contain enough design detail for bounded planning. If the epic contains only links and those links cannot be read, the system suppresses placeholder generation and returns an explicit context-unavailable warning.

3.6.2 Story Point Assignment. Initial estimates are derived from complexity priors:

$$\text{SP}(s) = f(\xi_{\text{scope}}, \xi_{\text{integration}}, \xi_{\text{risk}}, \xi_{\text{unknown}}), \quad (46)$$

where the four ξ terms are normalized complexity priors for scope, integration, risk, and unresolved uncertainty. The resulting score is mapped into Fibonacci-like buckets, such as 3, 5, and 8. In the current implementation, the defaults are deterministic (3/5) and could be learned in future variants.

Deterministic baseline is an intentional decision. It provides predictable behavior in early adoption phases and prevents estimation uncertainty until enough history is accumulated for learning. It also reflects a conservative evaluation approach in which the value provided by automation is evaluated against deterministic baseline behavior before more complex adaptive estimators are applied.

3.6.3 *Iterative Enrichment.* For each candidate story s :

- (1) run semantic analysis,
- (2) calculate readiness/confidence,
- (3) if $C(s) < \tau_c$, provide guidance and repeat the enrichment process within a bounded attempt budget,
- (4) provide a structured story payload.

Enrichment increases as each iteration brings explicit feedback from reviewers, such as missed edge cases, unresolved dependencies, or ambiguous non-functional requirements. Thus, the refinement process becomes a collaboration, not just prompting. This difference is important for proper human-AI team interaction [2, 6].

Moreover, in one embodiment, the output of the enrichment process follows a canonical, strongly typed schema: `problem_statement`, `actor_value`, `functional_scope`, `nfr_constraints`, `acceptance_criteria`, `dependency_map`, `risk_register`, and `test_considerations`. The strong typing increases interoperability of the result with review, analytics, and policy engines and eliminates ambiguity due to informality of the free-form text.

Coverage analysis at the epic level is possible via a capability matrix. Its rows include the capability dimensions like UI, API, data, observability, resiliency, and security, while its columns contain the proposed stories. The system is capable of identifying undercovered dimensions and producing the corresponding candidate stories. This changes the decomposition behavior from purely generative to constrained generation with well-defined goals.

3.6.4 *Ablation Execution Algorithm.* Algorithm 3 shows how the four modes are executed for the same epic unit. For each mode, the procedure enables only the components included in that condition, applies the same scoring schema, and returns measures of quality, efficiency, governance, composite performance, and assessment time. This common process supports a consistent comparison across B0, B1, B2, and B3.

Algorithm 3: Ablation-Mode Execution for B0/B1/B2/B3 Comparison

Input: Epic unit E_u , mode $m \in \{B0, B1, B2, B3\}$
Output: Metric row ($Q_m, \text{Eff}_m, G_m, \text{Composite}_m, \text{Time}_m$)

```

1 if  $m = B0$  then
2   | ExecuteManualReferenceCore( $E_u$ )
3 else
4   | ExecuteTemplateCore( $E_u$ ) if  $m = B1$ 
5   | ExecuteGenericLLMCore( $E_u$ ) if  $m = B2$ 
6   | ExecuteFullJiraEnhancerCore( $E_u$ ) if  $m = B3$ 
7 if  $m \neq B3$  then
8   | Disable components excluded from mode  $m$  (critic loop / context packer / validator / governed writeback)
9 Collect metrics using the same scorer and schema across all modes
10 Compute  $Q_m$  (quality),  $\text{Eff}_m$  (efficiency),  $G_m$  (governance),  $\text{Composite}_m$ , and  $\text{Time}_m$ 
11 return ( $Q_m, \text{Eff}_m, G_m, \text{Composite}_m, \text{Time}_m$ )

```

3.7 Policy and Governance Layer

The policy and governance layer controls whether a candidate story may move towards Jira creation. It checks confidence, available context, policy rules, validation, and human approval. The AI may generate and refine a story, but it cannot

approve the final writeback. First, the system checks whether a story needs additional human review:

$$I_{\text{esc}}(s) = \# [C(s) < \tau_e] \vee I_{\text{ctx}}(s). \quad (47)$$

where τ_e is the normalized escalation threshold and $I_{\text{ctx}}(s)$ denotes a partial external-context condition. This predicate cannot substitute the explicit approval indicator $A(s)$. Here, $I_{\text{esc}}(s) \in \{0, 1\}$ is the escalation indicator for candidate story s . It equals 1 when the confidence is below τ_e or when external context is incomplete. In this case, the story requires additional human review. Otherwise, it equals 0. Escalation does not replace the mandatory human-approval gate.

The system next checks whether candidate story s is eligible for creation:

$$g_{\Pi}(s) = \# [\Pi(s) = \text{ALLOW}], \quad (48)$$

$$I_{\text{create}}(s) = \# [C(s) \geq \tau_c \wedge A(s) = 1 \wedge V(s) = 1 \wedge g_{\Pi}(s) = 1 \wedge s \in \mathcal{F}_{\text{dup}}],$$

Here, $\#[\cdot]$ is an indicator function. It returns 1 when a condition is true and 0 otherwise. The term $\Pi(s)$ is the decision made by the active policy. Therefore, $g_{\Pi}(s) = 1$ means that the policy allows the story.

The creation rule contains five gates. First, $C(s)$ must reach the confidence threshold τ_c . Second, $A(s) = 1$ confirms human approval. Third, $V(s) = 1$ confirms that validation has passed. Fourth, $g_{\Pi}(s) = 1$ confirms policy permission. Finally, $s \in \mathcal{F}_{\text{dup}}$ confirms that the story is not a duplicate.

The value $I_{\text{create}}(s) = 1$ only when all five gates pass. If any gate fails, the story is not eligible for creation. This value indicates eligibility only. It does not mean that Jira has already been updated. The source-hash and idempotency checks must also pass, as shown in Algorithm 1. The conflict and target checks are applied by the writeback model described below.

Confidence and human approval are important gates. However, neither gate is sufficient by itself. Validation, policy permission, and duplicate control are also required. This design keeps the final decision with a reviewer and supports appropriate reliance on automation [2, 48].

Approval is always attached to a concrete execution run. This way, it can be evaluated in terms of the policy used during the actual execution of the decision, regardless of modifications made to the thresholds afterward. This traceability is crucial in regulated environments, where reproducibility of the process is sometimes more important than its result.

The governance layer can also impose approval criteria according to the reviewer's role: modifications to an external API contract should be approved by a domain owner, while stories affecting authentication flows need approval from a security reviewer. Such approval-routing criteria can be represented as policy predicates based on generated metadata and evidence tags.

3.8 Conflict-Safe Writeback Model

The previous section decides whether a candidate may enter the creation stage. It does not make the final change in Jira. In this section, *writeback* means that final Jira change. Approval and writeback may happen at different times. During this gap, the source issue, draft, policy, or target may change. A request may also be retried. The model below blocks stale changes and prevents duplicate creation.

We use two protection levels in this section. A2 combines a source-hash check with atomic idempotency. A3 adds a source-bound transaction envelope. These arrangements are evaluated later in Table 10.

Approval-bound context. An approval flag records the reviewer’s decision. However, it does not record the exact context that was approved. A3 therefore stores this context in an approved transaction envelope, denoted by $\mathcal{E}_a$:

$$\mathcal{E}_a = (r_{\text{run}}, i_{\text{issue}}, h_s, v_s, v_d, h_d, v_{\Pi}, h_{\Pi}, \mathcal{T}_{\text{target}}, a_{\text{dec}}, u_{\text{rev}}, t_a), \quad (49)$$

where r_{run} identifies the run and i_{issue} identifies the issue. The pair (h_s, v_s) records the approved source hash and revision. The pair (v_d, h_d) records the approved draft version and payload hash. The pair (v_{Π}, h_{Π}) records the policy version and snapshot hash. The set $\mathcal{T}_{\text{target}}$ lists the allowed Jira targets. Finally, a_{dec} , u_{rev} , and t_a record the approval result, reviewer, and time.

The system hashes the complete envelope to obtain the transaction identifier $I_{\text{tx}} = H(\mathcal{E}_a)$. The A3 idempotency key then binds this transaction to the issue:

$$K_{A3} = H(I_{\text{tx}}, i_{\text{issue}}). \quad (50)$$

Runtime validation. Immediately before writeback, the validator builds the runtime envelope $\mathcal{E}_t$. It has the same fields as $\mathcal{E}_a$. The validator rereads the current source, draft, policy, and target values. It carries forward the run, issue, and approval metadata from the approved envelope.

The system permits a mutation only when the two envelopes match and the decision is still APPROVE:

$$\text{permit}(\mathcal{E}_a, \mathcal{E}_t) = \mathbb{K}[\mathcal{E}_a = \mathcal{E}_t \wedge a_{\text{dec}} = \text{APPROVE}]. \quad (51)$$

An ABA change occurs when the source moves from state A to B and later returns to the same visible content A. Its revision has still changed. The envelope comparison detects this case. It also detects draft regeneration, policy drift, and target drift. A matching Jira text hash alone is therefore not sufficient.

Source-conflict check. The system also calculates a hash of the current normalized issue state:

$$h_{\text{current}} = H(\text{normalized issue state}), \quad (52)$$

It compares this value with the approved source hash $h_{\text{expected}} := h_s$. A mismatch moves the item to the conflict path, and Jira is not changed. Here, $\text{write}(s) = 1$ means that story s is written to Jira. Within the declared validator and fault model, the write authorization rule is:

$$\Pr(\text{write}(s) = 1 \mid h_{\text{current}} \neq h_{\text{expected}}) = 0. \quad (53)$$

Retry protection. The source check prevents a stale write, but a repeated request can still create a duplicate. A2 addresses this risk with a payload-level idempotency key:

$$K_{A2} = H(\text{normalized payload, target epic, policy version}). \quad (54)$$

Requests with the same normalized payload, target epic, and policy version receive the same A2 key. A3 uses the envelope-level key defined above. Since $I_{\text{tx}} = H(\mathcal{E}_a)$, its key is equivalently $H(H(\mathcal{E}_a), i_{\text{issue}})$. This binds the retry to the approved context.

The key is checked and reserved atomically before mutation. Hashing alone is not sufficient. Deterministic field ordering ensures that logically equal payloads produce the same hash input. Therefore, a repeated request can be handled as the same operation rather than as a new creation request. If the approved state has changed, the story must be validated and approved again. These guarantees apply within the implementation and fault model evaluated later in Table 10.

Per-story and batch outcomes. The previous rules decide whether one story may be written. A run may contain several candidate stories, so the system records each outcome separately. Let o_s denote the outcome of candidate s , where

$$o_s \in \{\text{CREATED}, \text{ALREADY_EXISTS}, \text{NOT_APPROVED}, \text{CREATE_FAILED}\}.$$

For a candidate batch $\mathcal{B}_{\text{batch}}$, the overall status follows this precedence rule:

$$\text{status}(\mathcal{B}_{\text{batch}}) = \begin{cases} \text{CREATED}, & \exists s \in \mathcal{B}_{\text{batch}} : o_s = \text{CREATED}, \\ \text{FAILED}, & \exists s \in \mathcal{B}_{\text{batch}} : o_s = \text{CREATE_FAILED}, \\ \text{ALREADY_EXISTS}, & \exists s \in \mathcal{B}_{\text{batch}} : o_s = \text{ALREADY_EXISTS}, \\ \text{FAILED}, & \text{otherwise.} \end{cases} \quad (55)$$

The conditions are read from top to bottom. A successful creation has the highest priority. If nothing is created, a creation error produces **FAILED**. If neither condition applies, a duplicate produces **ALREADY_EXISTS**. A batch containing only unapproved stories also produces **FAILED**. The detailed per-story results remain available in every case. The reviewer can therefore see each skipped item, its reason, any connector error, and the existing Jira key for a duplicate.

3.9 API and UI Innovation

3.9.1 Workflow API. The API supports two connected tasks. The first task enriches existing Jira items through a human review and revision loop. The second task decomposes an epic into candidate stories. Jira mutation remains separate from both tasks.

Run creation and inspection. POST `/api/v1/runs` starts an enrichment run for an issue, sprint, epic, or release. GET `/api/v1/runs/{runId}` returns the overall run status. GET `/api/v1/runs/{runId}/items/{issueKey}` returns the current draft. It also returns the evidence, warnings, readiness, confidence, policy state, and prompt-selection details. These read endpoints allow a reviewer to inspect the result before making a decision.

Human guidance and revision. The reviewer does not need to accept the first draft. POST `/api/v1/runs/{runId}/items/{issueKey}/approval` records a decision for one item. The allowed values are `approve`, `reject`, `regenerate`, `user_input`, and `prompt`. The `user_input` decision carries a human suggestion. The `prompt` decision carries a reviewer-written instruction for further modification. Both forms of guidance are sent back to the reasoning process, which produces a revised draft.

The run-level endpoint, POST `/api/v1/runs/{runId}/approval`, applies the same decision and guidance to every item in the run. The system may also ask a clarification question. In this case, POST `/api/v1/runs/{runId}/items/{issueKey}/interaction-answer` records the human answer and continues the revision. Human suggestions and follow-up prompts are therefore part of the API workflow.

Controlled writeback. POST `/api/v1/runs/{runId}/items/{issueKey}/writeback` writes one approved item after the policy, validation, source-conflict, and retry checks. POST `/api/v1/runs/{runId}/writeback` applies this operation across the run. Each item still receives its own result. This preserves item-level reasons for a successful, blocked, conflicting, or repeated request.

Epic story decomposition. POST /api/v1/epics/{epicKey}/stories/suggest reads the epic, existing child stories, and available evidence. It returns editable story candidates with readiness, confidence, threshold status, warnings, and evidence references. It does not create Jira stories.

POST /api/v1/epics/{epicKey}/stories/create receives the reviewed story payloads. It checks confidence and approval. It skips matching existing summaries and attempts to create only the remaining stories. Its response lists created and skipped items, including duplicate and creation-failure reasons.

The API therefore supports a full cycle: inspect, suggest, provide human guidance, revise, approve, and write back. Human users and controlled clients use the same contracts and governance checks.

3.9.2 Console Workflow. The console exposes the same workflow in three connected views. The enrichment view starts a run. It shows each Jira draft with its evidence, warnings, readiness, confidence, and selected prompt. The review view allows the user to approve, reject, request regeneration, provide a human suggestion, or submit a Jira-specific prompt for further modification. The same guidance can be applied to one Jira or to all Jira items in the run.

The **Epic Story Breakdown** view provides the following features:

- (1) epic-key and project-key inputs,
- (2) the suggestion action, which reads the epic and proposes child stories,
- (3) an editable structured payload for reviewing each story, with an approved flag per story,
- (4) the approved-story creation action for writing back qualified rows.

The reviewer can edit the generated content, add missing context, or provide a new prompt before approval. Writeback remains a separate action. This prevents a revision request from being treated as permission to change Jira.

Human-led and agent-led operations are both provided via the API. The human reviewer evaluates narrative quality and potential risks via the console, while controlled agents or pipelines use the same endpoints based on deterministic payload contracts. This allows governed integration with release-planning bots, PM dashboards, and governance data lakes.

The UI focuses on “decision visibility,” giving the reviewer the sources of confidence, warnings, policy reason codes, approval provenance, reasons for skipping duplicates, and preserved retry selections for each story candidate. This discourages reviewers from treating confidence as an unsupported number and enables quicker and better accept, reject, or retry decisions. The design seeks appropriate trust rather than maximum automation, reflecting evidence that human-AI team performance depends on users’ mental models of system behavior [2, 6, 33].

4 EVALUATION DESIGN AND EVIDENCE

Evaluation of evidence is classified by the way evidence is acquired and the claims it supports. This ensures that field observations, controlled fault injections, offline replays, and generated sensitivity analyses are not treated as equivalent empirical samples. Table 4 specifies the units of analysis and inference limits for each evidence block.

For reproducibility, the evidence units are organized around story-level endpoints, confidence and readiness summaries, policy decisions, and connector outcomes. The fault-injection units use a fixed random seed and the same transaction schedules for each arrangement. In turn, the replay and generated-unit blocks use their corresponding seeds and transformations. In accordance with empirical software-engineering guidelines on clarity of context, measures, and analysis procedures, we provide the reporting details in this format [31, 51, 56, 65].

Table 4. Evidence hierarchy and claim boundaries. “Generated” denotes deterministic formula-based perturbation or scoring, not an observed execution of the named baseline.

Evidence block	Unit and scale	Primary use	Explicit inference boundary
Authenticated Jira field trace	70 story endpoints; two waves; 14 epics	RQ1 workflow feasibility and RQ2 endpoint telemetry	No randomized control, rejected case, or downstream delivery outcome; independent content ratings are reported separately.
Authenticated-output artifact assessment	Two independent reviewers; 69 full-text stories; 14 epic packages; three masked repeats per reviewer	RQ1 artifact quality and RQ2 re-viewability	B3-only absolute assessment; no baseline comparison, downstream outcome, or cross-organization generalization.
Matched blinded baseline assessment	Five reviewers; 42 primary packages (14 inputs $\times$ B1/B2/B3); 210 stories; four masked repeats per reviewer	RQ4 package quality, readiness, and reviewer-recorded effort	Reviewers 3–5 form the primary analysis; one portfolio, no manual baseline, no downstream outcome, and no empirical method-origin guess.
Controlled mutation-safety ablation	Nine scenarios; 2,000 payloads/scenario; four arrangements; 72,000 method-level observations	RQ3 stale-state, retry, and duplicate handling	Scenario-balanced fault injection; rates are not estimates of production incident prevalence.
Offline bandit replay	1,000 epochs anchored to 70 terminal outcomes	Numerical posterior-update and stopping diagnostics	Not 1,000 live Jira mutations and not causal evidence of policy effectiveness.
Internal generated-unit sensitivity analysis	5,000 formula-derived units per method	Stress-test the declared scoring and composite-index model	B0–B3 metrics are generated; this is not a measured manual-versus-LLM experiment.
Public issue-text sensitivity analysis	300 SWE-bench Lite issue records	Test whether the scoring pipeline accepts public issue text	No patches are generated and no SWE-bench tests are executed; this is not a resolution benchmark result.

5 RESULTS AND COMPARATIVE ANALYSIS

5.1 Authenticated Jira Deployment Evidence (RQ1–RQ2)

Figure 5 presents the authenticated endpoint summary and generated-unit raincloud side by side while stating their evidence status explicitly. Panel (a) is based on the 70 live Jira story endpoints observed during two authenticated deployment waves. It reports story-level terminal confidence rather than aggregating the data into the older nine-row issue summary, preserving the paired pre- and post-endpoints used in the descriptive and bootstrap analyses below. Panel (b) presents the formula-derived sensitivity distribution for 5,000 generated units and is not treated as live comparative evidence.

Figure 5(a) summarizes the distribution of terminal endpoints. Each arc represents one Jira story, with longer arcs indicating higher displayed confidence C_{10} on the zero-to-ten scale; color identifies the selected refinement strategy. All 70 records reached a governed terminal path and ended at or above $C_{10} = 9$. Because neither deployment wave retained rejected or below-threshold stories, the chart shows successful-path feasibility rather than field-wide acceptance performance. Panel (b) gives a succinct distributional check of the declared composite-index formulas. The separation among B0–B3 is induced by the scoring model and seeded perturbations, and thus should not be interpreted as measured superiority over executed baselines.

For adaptive-policy telemetry, the current evidence base consists of two authenticated and verified Jira deployment waves: a five-epic wave with 25 terminal story records and a nine-epic Confluence-derived wave with 45 terminal story records. Together, the waves provide a total of 70 persisted endpoint pairs containing confidence, shaped reward, loss, lag, selected prompt strategy, approval, and writeback status. Each current endpoint has one terminal attempt index, while intermediate live refinement attempts were not persisted. Hence, the endpoint data support paired confidence analysis, but do not support an observed multi-step learning-curve claim.

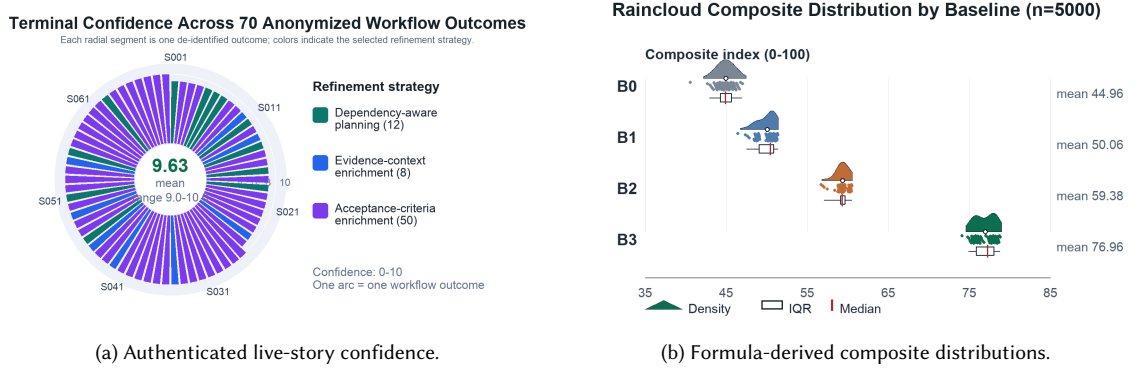


Fig. 5. Two performance views with different evidence status. (a) Terminal confidence for all 70 authenticated live Jira story outcomes. Each radial segment represents one story, ordered by terminal confidence and colored by the selected refinement strategy. (b) Raincloud distribution of formula-derived composite indices over 5,000 generated units per method, showing density, sampled values, IQR, median, and mean. Panel (b) is a sensitivity output of the scoring model, not an observation of comparative performance.

Table 5. Accumulated live Jira evidence across two authenticated deployment waves. The table distinguishes the observed production-style outcome signal from the larger comparative simulation and public-benchmark evidence blocks.

Evidence dimension	Observed value	Interpretation
Cumulative scope over life-time	2 authenticated and validated waves, 70 story outcomes	Consists of one previous five-epic deployment wave (25 outcomes) and one subsequent nine-epic Confluence-derived AES deployment wave (45 outcomes).
Governing terminal conditions	70/70 (100.0%); 69 created, 1 duplicate-safe existing record	All candidates reached a governed terminal state, and the existing record was validated without creating a duplicate.
Reward distribution	range 0.40–0.70; mean 0.5171; cumulative 36.20	The reward is not a static indicator of success; it depends on confidence lift and whether writeback occurred.
Loss from terminal confidence gap	mean 0.00	The normalized difference from target confidence was 0 for all persisted outcomes; it is not an estimate of sequential training loss.
Refinement strategy distribution	Dependency-aware planning: 12, Evidence-context enrichment: 8, Acceptance-criteria enrichment: 50	The evidence covers several policy-governed refinement strategies rather than one hard-coded prompt path.

The latest validated wave is shown separately because it traversed the complete Confluence-to-Jira path involving nine additional epics. Table 6 gives the statistics for this wave: Jira Enhancer produced or verified 45 story records, created 44 new stories, detected one pre-existing story through duplicate-safe link checking, and associated all 45 story records with the intended parent epic. All 44 newly created payloads contained the four audited elements: an external evidence reference, acceptance criteria, test criteria, and operational-observability criteria. The pre-existing story is excluded from the payload denominator because its generated payload was unavailable for the same audit.

Table 7 gives the story-level uncertainty based on 50,000 fixed-seed bootstrap resamples [16]; binary connector outcomes are evaluated with Wilson intervals. Mean confidence increased from 7.46 (SD 0.56) to 9.63 (SD 0.49), a paired mean increase of 2.17 points (95% bootstrap CI [1.94, 2.40]), while the median difference was 2.50 (IQR 1.00–3.00). All 70 changes were positive (exact two-sided sign test, $p = 1.69 \times 10^{-21}$). This shows consistent movement of the

Table 6. Most recent authenticated live batch derived from Confluence. The table reports the 45-story expansion that followed the earlier 25-story run and forms part of the cumulative 70-outcome evidence base.

Evidence dimension	Observed value	Interpretation
Deployment scope	9 epics, 45 stories	Exercises the full Confluence-to-Jira planning path at batch level.
Writeback outcome	44 created; 1 duplicate-safe link check	Distinguishes new issue creation from reuse of an existing story.
Parent-epic binding	45/45 (100.0%)	Confirms that every generated or checked story is attached to its intended epic.
Generated-payload coverage	44/44 new stories (100.0%)	All newly created payloads include the evidence reference, acceptance criteria, tests, and operational criteria; the pre-existing linked record is excluded.
RL outcome telemetry	45 outcomes; reward 0.50–0.70; mean 0.5822	Rewards vary with confidence lift instead of treating all accepted stories as a flat success.
Refinement-strategy signal	Dependency-aware planning: 4, Evidence-context enrichment: 5, Acceptance-criteria enrichment: 36	Shows which policy-selected refinement strategies were used in the live batch.
Convergence signal	mean terminal loss 0.00	The confidence-gap objective reached zero for the recorded live outcomes.

Table 7. Observed live endpoint and connector statistics. Confidence intervals quantify sampling uncertainty within the recorded story set; they do not remove portfolio selection or within-epic dependence.

Observed measure	Estimate	95% interval	Inference boundary
Paired confidence change	+2.17 points ($n = 70$)	[1.94, 2.40] bootstrap CI	System telemetry; not an independent human rating of story quality.
Positive confidence change	70/70 (100.0%)	[94.8%, 100.0%] Wilson CI	Paired endpoint diagnostic; all rows were retained and no zero deltas occurred.
New Jira issue creation	69/69 (100.0%)	[94.7%, 100.0%] Wilson CI	Conditional on approved, non-duplicate candidates in the two recorded waves.
Latest-wave parent binding	45/45 (100.0%)	[92.1%, 100.0%] Wilson CI	Direct connector outcome for nine epics; no cross-project generalization is implied.
Generated-payload criteria coverage	44/44 (100.0%)	[92.0%, 100.0%] Wilson CI	The pre-existing duplicate-safe record is excluded because its generated payload was unavailable.

system-derived score toward its configured threshold. The blinded review offers a separate assessment of content quality, but confidence remains a system-derived metric rather than a calibrated probability of successful deployment.

5.2 Blinded Independent Artifact Assessment (RQ1–RQ2)

RQ1: How can a human-AI workflow split underspecified software epics into reviewable implementation stories without compromising human authority over acceptance and writeback?

RQ2: How can machine confidence, evidence provenance, and policy gates be combined to help reviewers determine whether AI-generated planning records are reliable enough for action?

Two software-engineering reviewers independently assessed anonymized planning artifacts from the authenticated waves. The review materials omitted method labels, Jira and epic identifiers, internal links, author fields, confidence, readiness, reward, loss, policy-arm values, timestamps, and writeback status. Artifact order was randomized independently for each reviewer. All 70 outcomes remained in scope: 69 stories with full payloads received numerical ratings, while the one pre-existing summary-only record was retained for accounting and marked not rateable. Each reviewer also assessed all 14 five-story epic packages. Three masked repeats per reviewer were excluded from the primary estimates and retained only for descriptive intra-rater checks.

The prespecified story rubric used six dimensions on a 1–5 anchored scale: scope fit, clarity, acceptance/testability, dependency/risk treatment, granularity/actionability, and implementation readiness. The package rubric used coverage breadth, non-redundancy, decomposition coherence, dependency sequencing, and overall readiness. Reviewer scores were first averaged within each artifact. The primary story interval used 50,000 bootstrap resamples of complete epic clusters, preserving the dependence among stories from the same epic.

Table 8. Independent blinded ratings on the prespecified 1–5 scale (69 full-text stories and 14 epic packages).

Measure	Mean
Story composite	3.98
Scope fit	4.97
Clarity	3.90
Acceptance and testability	3.62
Dependency and risk treatment	3.62
Granularity and actionability	4.26
Implementation readiness	3.54
Epic-package composite	4.41
Coverage breadth	4.86
Non-redundancy	4.79
Decomposition coherence	4.75
Dependency sequencing	4.04
Overall package readiness	3.64

The mean story composite was 3.98/5 (epic-clustered 95% bootstrap CI [3.85, 4.11]). When story composites were aggregated within each epic, all 14 epic means exceeded the neutral midpoint of 3 (two-sided Wilcoxon signed-rank test [63], $p = 0.00098$). The package composite mean was 4.41/5, and all 14 package composites also exceeded 3 ($p = 0.00087$). Neither reviewer recorded an unsupported claim in a rateable story. The numerical profile is more informative than a single readiness label: scope fit and epic-level coverage were high, whereas acceptance/testability, dependency/risk treatment, and implementation readiness retained more room for reviewer-guided refinement.

Absolute-agreement ICC(2,1) was 0.45 for the story composite and 0.58 for the package composite. Exact agreement across story dimensions ranged from 30.4% to 94.2%, while agreement within one scale point ranged from 91.3% to 100%; every package dimension was within one point in all cases. Some weighted kappa values were near zero despite high observed agreement because variability was low near the top of the scale. We therefore report exact agreement, agreement within one point, and ICC together with kappa rather than relying on kappa alone. Both reviewers reproduced the same composite score for all three masked repeats, although three repeated pairs per reviewer provide only a limited check of consistency.

The reviewers used different cutoffs for categorical decisions. Reviewer 1 labeled 61 stories “Minor revision” and 8 “Major revision” stories, whereas Reviewer 2 labeled 14 stories “Ready,” 48 “Minor revision,” and 7 “Major revision.” We thus do not aggregate these categories into an overall acceptance rate or state unanimity of readiness. An appropriate interpretation is that the system produced consistent, well-decomposed packages and high-quality artifacts in general, and human review was beneficial for criteria clarification, dependencies, risks, and execution particulars. This block examined only Jira Enhancer outputs; thus, it did not compare the system against manual, template-based, or generic-LLM baselines.

5.3 Matched Blinded Baseline Comparison (RQ4)

RQ4: How does a governed human-AI workflow perform, relative to executable template/rule and generic-LLM baselines, on decomposition quality, review effort, and implementation readiness?

For the matched experiment, we held frozen the same 14 de-identified epic and evidence packages for the three methods. Each method produced five stories per package, according to a common schema. B1 ran the deterministic evidence-section parser and candidate builder. When the parser returned fewer than five stories, a declared static completion rule filled the remaining slots; this occurred for 24 of the 70 B1 stories. B2 and B3 passed through the authenticated OpenAI Codex service using model gpt-5.6-sol and the same evidence limitation. B2 made one generic generation request per epic, without a critic pass, confidence feedback, policy gate, retries, or reviewer fixes. B3 initially planned the epic and subsequently used Jira Enhancer’s story-level enrichment with the bounded critic loop, scoring, and policy validation. Human approval and Jira writeback were off for each method.

Resource consumption is part of the treatment definition. B1 made zero model requests. B2 made 14 successful requests, one for each package, while B3 made 173: 14 planning requests and 159 story-enrichment attempts. The total execution times were 2,087 seconds for B2 and 19,906 seconds for B3. As all input elements were processed simultaneously, these are the times when input elements are exposed to the model call. It is the comparison of pipeline completeness, quality, and reviewability; it cannot be seen as a compute-normalized efficiency comparison.

Five reviewers each received an independently randomized workbook containing 42 primary packages and four variants based on masked repetitions. The variants based on the same input element were separated by at least ten rating rows. All workbooks masked method identity, model and runtime information, source IDs, links, confidence, policy status, and authorship. The reviewers scored, using the anchored 1–5 scale, scope correctness, coverage completeness, story uniqueness, acceptance and testability, dependency and risk handling, granularity and executability, and implementation readiness. They also indicated disposition, confidence, actual time spent on assessment, the number of substantial modifications performed, and an evidence-related comment. Because Reviewers 1 and 2 had previously seen B3-only material, the prespecified primary analysis uses Reviewers 3–5. The result from all five reviewers is reported as a sensitivity analysis, and masked repeats are excluded from every method-effect estimate.

We treated the epic, rather than an individual rating or story, as the unit of inference. Within each input-method cell, scores were averaged across the three primary reviewers, leaving 14 matched observations per method. A crossed random-effects estimate was uninformative because between-reviewer score variance was near zero and only 14 epic clusters were available. We therefore used the robust alternative specified in the protocol: a 200,000-draw within-epic permutation test for the three-method omnibus comparison; exact paired sign-flip tests for the prespecified B3–B2 and B3–B1 contrasts; 100,000-draw epic-bootstrap confidence intervals [16]; Holm correction [23] within each outcome family; and paired rank-biserial effect sizes. Review time and edit counts were examined with the same matched, distribution-free procedure. The omnibus composite difference was detectable ($Q_{\text{omni}} = 19.75$, Monte Carlo $p < 10^{-5}$).

Table 9. Matched blinded review for the primary reviewer set (Reviewers 3–5): method summaries and prespecified package-composite contrasts. Scores use the anchored 1–5 scale; time and edit counts are medians [IQR]. Confidence intervals resample the 14 matched epics, and exact sign-flip tests are Holm-adjusted across the two contrasts.

Panel A: Method summaries					
Method	Composite [95% CI]	Readiness	Time	Edits	Ready/minor
B1	2.74 [2.45, 3.04]	2.50	9.0 [8.0, 10.0]	10.5 [7.0, 14.0]	50.0%
B2	4.21 [3.88, 4.53]	3.52	10.0 [10.0, 11.8]	2.5 [1.0, 5.0]	66.7%
B3	4.31 [4.09, 4.51]	3.74	9.0 [9.0, 10.0]	2.0 [2.0, 3.0]	88.1%

Panel B: Prespecified package-composite contrasts					
Contrast	Mean difference [95% CI]	Raw p	Holm p	r_{rb}	+/-/=
B3–B2	0.10 [−0.13, 0.36]	0.4854	0.4854	0.12	4/7/3
B3–B1	1.56 [1.23, 1.91]	0.0001	0.0002	1.00	14/0/0

Under the applicable IIT Roorkee and Oracle processes, the study owner determined that this internal, low-risk artifact-rating activity did not require separate consent or institutional approval. Reviewers received the study instructions before they returned their completed workbooks. The workbooks used reviewer codes and collected no names, demographic information, sensitive attributes, or other personal identifiers. The de-identified input packets remained access-controlled.

The prespecified artifact-only contrast between B3 and B2 was +0.10 points on the 1–5 composite scale (95% CI [−0.13, 0.36], Holm $p = 0.485$, $r_{rb} = 0.12$). B3 scored higher on four epics, lower on seven, and tied on three. This comparison is limited to the visible story packages; policy, approval, recovery, and writeback are not considered here. Against B1, B3 was higher on all 14 epics, with a mean difference of 1.56 points [1.23, 1.91] ($p_{\text{Holm}} = 0.00024$, $r_{rb} = 1.00$). The five-reviewer sensitivity analysis showed the same pattern: B3–B2 was +0.09 [−0.13, 0.35] ($p_{\text{Holm}} = 0.505$), whereas B3–B1 was +1.58 [1.25, 1.92] ($p_{\text{Holm}} = 0.00024$).

At the level of dimensions, this analysis provides some insight into the near-zero B3–B2 composite-score difference. Descriptively, B3 was rated better for granularity/actionability (+0.55), dependency/risk treatment (+0.29), implementation readiness (+0.21), story distinctness (+0.14), and scope fidelity (+0.07). B3 was rated worse on acceptance/testability (−0.43) and coverage completeness (−0.14). None of these seven exploratory comparisons was significant after Holm correction. Also, the dimensions do not capture B3’s contribution to governance. At best, the above comparisons indicate the move toward greater boundness, awareness of dependencies, and orientation toward implementation.

Reviewers took 1.02 fewer minutes, on average, to assess a B3 package than a B2 package (95% CI [−1.57, −0.48], $p_{\text{Holm}} = 0.0098$, $r_{rb} = -0.78$). The estimate of the number of edits required was 0.64 smaller for B3, even though the interval included no difference [−1.55, 0.19] ($p_{\text{Holm}} = 0.205$). Compared to B1, B3 required 8.02 fewer edits [−9.90, −6.21] ($p_{\text{Holm}} = 0.00024$). Assessment time did not differ significantly. The descriptively better disposition was also observed: 88.1% of B3 ratings were Ready or Minor revision, compared to 66.7% of B2 ratings and 50.0% of B1 ratings. We do not pool these labels into a causal acceptance probability.

Agreement among the primary reviewers was very high. Composite ICC(2,1) was 0.999, exact pairwise agreement on the composite was 96.8%, and all pairwise differences were no more than one point. Among the 12 primary masked-repeat pairs, all 84 dimension ratings and dispositions were identical. However, each package received three distinct narrative comments, and mean pairwise text similarity was only 0.127. We present both facts. While close scores may

be due to a tightly anchored rubric and large differences between the packages, four repeats per reviewer are too few to prove general interreviewer substitutability.

These findings give a bounded answer to RQ4. Jira Enhancer produced substantially stronger packages than the deterministic template/rule baseline and required fewer edits. B2, however, is a text-only condition that stops after one-shot drafting; it does not include B3's provenance, refinement, policy, approval, recovery, or mutation controls. The composite artifact test did not detect a B3–B2 difference. Even so, B3 required significantly less assessment time and received 5, rather than 14, Major revision decisions across 42 primary ratings. The supported advantage is therefore at workflow level: lower reviewer effort, together with governance and mutation controls that B2 does not provide. The study does not establish a general advantage in prose quality, end-to-end productivity, compute efficiency, downstream quality, or cross-domain transfer.

5.4 Controlled Mutation-Safety Fault Injection (RQ3)

RQ3: What failure modes emerge when agentic planning systems interact with enterprise issue trackers, including missing evidence, duplicate story creation, partial writeback failure, and stale approvals?

We tested the writeback boundary independently of text generation. The experiment applies the same payload and fault schedule to four arrangements: A0 ordinary approval plus logging; A1 source-hash conditional write; A2 source hash plus atomic idempotency; and A3 a source-bound transaction envelope that additionally binds source revision, approved draft version and hash, policy snapshot, reviewer decision, and target set. The nine scenarios are valid control, source edit, source ABA, draft drift, policy drift, target drift, response-loss retry, concurrent retry, and combined source/draft drift. Each scenario contains 2,000 distinct payloads, giving 18,000 transactions for each arrangement and 72,000 method-level observations in total.

A3 blocked every injected stale mutation and condensed retry duplicates into a single remote mutation while preserving every eligible write. Comparison to A2 emphasizes the effect of tying approval to more than the current state of the source content. Hashing only the source cannot detect ABA revisions, regeneration of drafts, policy change, or target-set change, when the Jira text has not changed. Seven implementation tests demonstrate correct envelope construction, stale-approval prevention, and idempotence allowance. These tests show how stale-approval and idempotence work given the declared fault model. They do not assess the frequency or effect in production.

Figure 6(a) needs careful interpretation. Since the live file contains just one before-and-after snapshot pair for each story, the 16 points in the figure come from the exponential interpolation declared in the paper, not from 16 observed interventions for each story. Among the 70 terminal records, shaped reward ranges from 0.40 to 0.70, with mean 0.5171 and sum 36.20, while endpoint loss is zero by construction once the target is reached. The Laplace-domain representation establishes stability of the assumed lag model when $\kappa > 0$, but it does not prove either global optimality of the prompt policy or independent validity of the confidence score. The observed endpoints are consistent with positive gain under the model, but longitudinal attempt logs are required to validate the continuous-time approximation.

Figure 6(b) is not a substitute for 1,000 live Jira mutations. It is a purely offline numerical experiment that samples outcomes from the confidence-shaping model based on 70 terminal records. After 1,000 epochs, the replayed agent obtained cumulative reward 576.44, with a final 50-epoch rolling reward of 0.5837 and a final 50-epoch rolling loss of 0.00. All three refinement strategies crossed $C_{10} = 8$ and reached $C_{10} = 9.00$ or higher. The stated convergence epochs were 24 for dependency-aware planning, 36 for evidence-context enrichment, and 58 for acceptance-criteria enrichment. Their final posterior means were 0.7843, 0.7847, and 0.7905, respectively, with balanced selection shares. These statistics support the numerical properties of the replay design under its reward scheme, but they do not establish convergence

Table 10. Controlled mutation-safety ablation. Rates use an equal mixture of the nine declared scenarios and therefore measure coverage under the injected schedule, not production prevalence. Eligible-write success is computed over valid-control and retry scenarios.

Arrangement	Stale mutation	Duplicate mutation	Any unsafe mutation	Correct outcome	Eligible-write success
A0 ordinary ap-proval/log	66.7%	33.3%	88.9%	11.1%	33.3%
A1 source-hash conditional	44.4%	22.2%	66.7%	33.3%	33.3%
A2 source hash + atomic idempotency	44.4%	0.0%	44.4%	55.6%	100.0%
A3 source-bound transaction envelope	0.0%	0.0%	0.0%	100.0%	100.0%

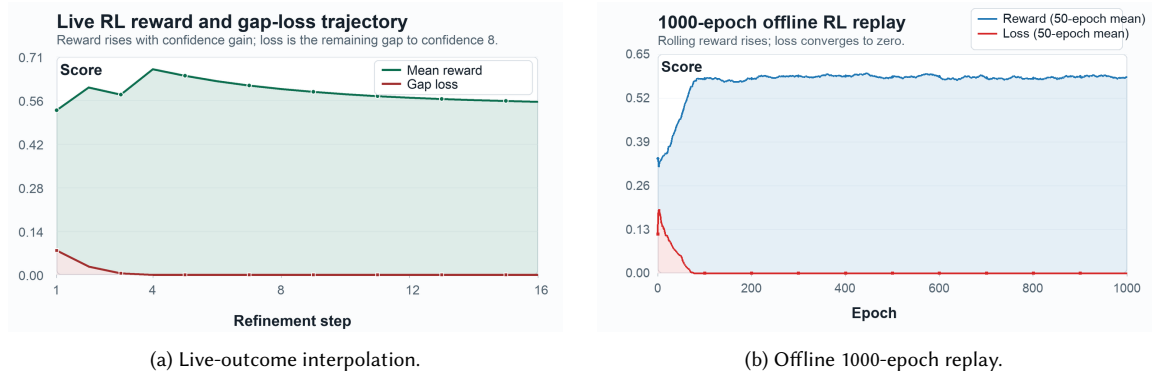


Fig. 6. Reward/loss analysis in two complementary views. (a) Analytical interpolation of shaped reward and confidence-gap loss between the 70 observed before/after endpoints. Fractional credit is assigned for confidence gain, and convergence credit is added after displayed confidence reaches $C_{10} = 8$ (normalized shaping threshold $\tau_r = 0.8$). Intermediate steps are model-implied rather than persisted live attempts. (b) Offline 1000-epoch Thompson-sampling replay based on the same 70 observed live RL outcomes. The replay uses shaped reward: fractional credit is assigned for confidence gain, convergence credit is added once displayed confidence reaches $C_{10} = 8$ ($\tau_r = 0.8$), and loss is the remaining normalized gap to that shaping threshold.

or a causal benefit of the implemented policy. Such claims require evidence from observed sequential attempts and varied terminal outcomes, including rejection, regeneration, conflict, and low confidence.

Figure 7(a) shows an older decision-only proxy trace rather than the 70 terminal live records. The figure groups confidence before and after refinement by attempt index, with denominators decreasing from 17 and 16 observations at attempts 1 and 2, respectively, to only two or three at later attempts. It is therefore a descriptive diagnostic of the retained trace, not a balanced longitudinal estimate.

At attempt 2, mean confidence changed from 2.00 to 7.69 across 16 retained proxy observations. Several later attempt indices show little change, but their much smaller denominators prevent this pattern from being attributed to prompt quality or diminishing returns. Future runs should retain every attempt for every story and analyze the repeated measures with clustering at story and epic levels.

Taken together, the live block establishes authenticated workflow feasibility and provides paired endpoint telemetry. It does not yet include post-delivery success labels; accordingly, ECE and Brier score are defined in Section 3.4 but are

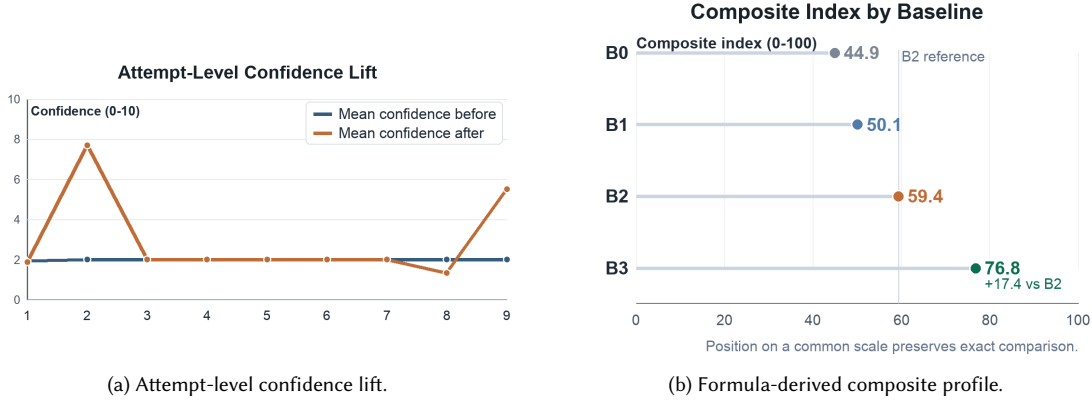


Fig. 7. Diagnostics with different evidence status. (a) Confidence lift for the older decision-only proxy trace. Pre- and post-refinement means are grouped by attempt index; sample sizes differ by attempt and are not representative of the 70-story live endpoint sample. (b) Composite-index comparison in the formula-generated sensitivity model. B0 is the equation-based manual reference, B1 is the lexical template/rule core, B2 is the generic-LLM-style heuristic core that does not use an LLM, and B3 is the equation-based profile informed by the observed Jira Enhancer confidence and readiness.

not estimated using approval as a surrogate outcome because doing so would conflate governance acceptance with delivered quality.

6 GENERALIZABILITY BOUNDARIES AND WORKFLOW TRADE-OFFS

6.1 Simulation-Based Comparative Sensitivity Analysis (B0–B3)

The B0–B3 block represents a deterministic stress test of the design space rather than four executions of the observed method. B0 generates a formulaic reference. B1 uses lexical rules and fixed metric equations. B2 analyzes the issue text and generates a generic-LLM-style profile using fixed heuristics, without calling a language model. B3 uses the observed Jira Enhancer confidence and readiness as anchor values, whereas its coverage, time, edit, rework, and governance values are computed from declared formulas. Each row indicates the run-item and intermediate features used by each core.

This diagnostic block checks whether the scoring and composite-index models remain stable when the observed runtime units are perturbed in a controlled fashion. It allows sensitivity analysis, mechanism analysis, and implementation regression testing. Nevertheless, it cannot measure the performance of manual teams or generic LLMs, nor it can evaluate the causal effect of Jira Enhancer on that performance [65]. The matching experiment mentioned above gives the independently executed B1, B2, and B3 artifacts required to answer RQ4; the produced profiles are still different from the observed null B3–B2 quality contrast.

Figure 7 contains two compact diagnostics next to each other while keeping their different evidentiary value. Panel (a) is the decision-only proxy trace, and Panel (b) is the formula-derived sensitivity profile.

The formulas defined in Figure 7(b) produce composite mean scores of B0=44.96, B1=50.06, B2=59.38, and B3=76.96. In the model, B3 profile beats B2 by 17.58 points. The difference comes from the model assumptions and weights; it is not an observed lift over an executed LLM baseline.

Table 11. Comparison of failure-mode coverage. This table presents a mechanism-oriented analysis derived from implemented behavior, not a measured frequency table. It identifies the collaboration failures explicitly represented and recoverable in each approach.

Failure mode	B0 manual	B1 template/rule	B2 generic LLM	B3 Jira Enhancer
Missing external evidence	Manual delay or meeting follow-up	No evidence-aware handling	Risk of fluent but weakly grounded output	Warning codes plus embedded Jira-context fallback when substantive design text exists
Duplicate story creation	Depends on reviewer memory	Weak summary matching only	Possible if generated wording changes	Deterministic normalized-summary skip with existing issue key reported
Partial create failure	Manual reconciliation	No preserved decision state	No structured partial outcome	Per-story create_failed status, skipped list, and retry-preserved UI state
Stale approval after tracked edit	Human process risk	Not modeled	Not modeled	Source-hash check blocks writeback and forces revalidation
Unsafe high-confidence output	Reviewer judgment only	Static guardrails	Confidence may be uncalibrated	Confidence threshold, validation, policy gate, and explicit approval key

6.2 Why B3 Is the Stronger End-to-End Workflow

B2 is a single-run text generation control, whereas B3 runs the same input via evidence packing, bounded refinement, confidence scoring, policy evaluation, audit persistence, approval binding, recovery state, and conflict-safe writeback. As the blinded artifact scores do not account for this set of mechanisms, they cannot independently confirm end-to-end workflow superiority.

On the common inputs, reviewers assessed B3 packages 1.02 minutes faster than B2 packages (Holm-adjusted exact $p = 0.0098$, paired rank-biserial $r_{rb} = -0.78$). The descriptive statistics demonstrate the same trend: 37 out of 42 B3 scores (88.1%) were Ready or Minor revision, as opposed to 28 of 42 B2 scores (66.7%), and the counts of Major revision decisions were 5 and 14 for B3 and B2, respectively. The artifact-only composite difference stays non-significant; thus, it cannot be used as evidence of better prose produced by B3. The workflow hypothesis also relies on the mechanism and fault evidence. Table 11 presents the controls available in B3 and missing in B2, and Table 10 reports the lack of unsafe mutations and 100% eligible-write success rate for B3's source-bound transaction mechanism. B3 is the stronger evaluated workflow within the declared test suite: it takes less review time and provides governance, recovery, and fault-tested mutation controls.

Table 11 captures the implemented mechanism-coverage model. It is not a frequency distribution; the controlled fault-injection experiment in Table 10 provides evidence for RQ3.

6.3 Mechanisms Encoded in the B3 Profile

The generated B3 profile, thus, involves a stacked set of mechanisms instead of making a single prompt choice:

- (1) **Critic loop driven by rubrics with threshold criteria:** candidate drafts get iteratively revised until the target thresholds of clarity, testability, dependency, evidentiary basis, and actionability are achieved.
- (2) **Context packing rather than evidence passthrough:** the model receives ranked and compact context on issues rather than large and noisy blocks of evidence.
- (3) **Pre-policy evaluation checks for contradictions and gaps** prevent fluent but unsafe outputs from being marked ready.
- (4) **Governance-aware scoring:** weak consistency or rubric signals reduce confidence and readiness scores.
- (5) **Policy-controlled external mutation:** even a candidate draft with a high readiness score will have to be evaluated and approved for any external mutations.

These mechanisms form the basis for the B3 scoring assumptions and are present in the implementation, but the generated comparison cannot identify them as separate sources of the modeled difference. A measured ablation would disable one mechanism at a time on the same inputs, beginning with the critic loop and then proceeding through context packing, the validator, and policy coupling. The thematic background analysis and Table 1 connect findings from retrieval, reasoning, tool orchestration, and governance research to the corresponding B3 mechanisms.

6.4 Generated-Unit Stress Test (5,000 Units)

Generator stress testing proceeds by cycling the generator through observed runtime units under deterministic seeded perturbations until 5,000 units have been generated, after which the B0–B3 scoring formulas are evaluated on the generated units. The generated dataset includes item-level features, summary statistics, and high-lift cases, but does not produce 5,000 independent epics.

The method-level composite distributions are:

- **B0 manual:** mean 44.96, std 1.05.
- **B1 template rules:** mean 50.06, std 1.07.
- **B2 generic LLM:** mean 59.38, std 0.72.
- **B3 Jira Enhancer:** mean 76.96, std 1.22.

For each method m ($n = 5,000$ units per method), the composite index is calculated as a bounded weighted sum:

$$\text{Composite}_m = w_Q Q_m + w_{\text{Eff}} \text{Eff}_m + w_G G_m, \quad w_Q + w_{\text{Eff}} + w_G = 1, \quad w_Q, w_{\text{Eff}}, w_G \geq 0. \quad (56)$$

Equal weighting of all baselines in the reported experiments results in score differences due to method performance and not metric weightings. Let $\Delta_{B3,m} = \bar{X}_{B3} - \bar{X}_m$, where $m \in \{B0, B1, B2\}$ denotes a baseline. All subsequent confidence intervals and hypothesis tests are performed on this composite variable. Figure 5(b) gives a compact raincloud view of the generated scores. Figures 8a–8c provide three larger diagnostic views. Each view answers a different question about the scoring formulas.

The violin plot in Figure 8a shows the shape and spread of each generated distribution. The median and interquartile range are 44.95 [44.26, 45.69] for B0, 50.45 [49.04, 50.89] for B1, 59.33 [59.09, 59.69] for B2, and 77.27 [75.83, 78.05] for B3. The narrow ranges show that the seeded perturbations preserve the separation encoded by the formulas. They do not show variation across independent projects.

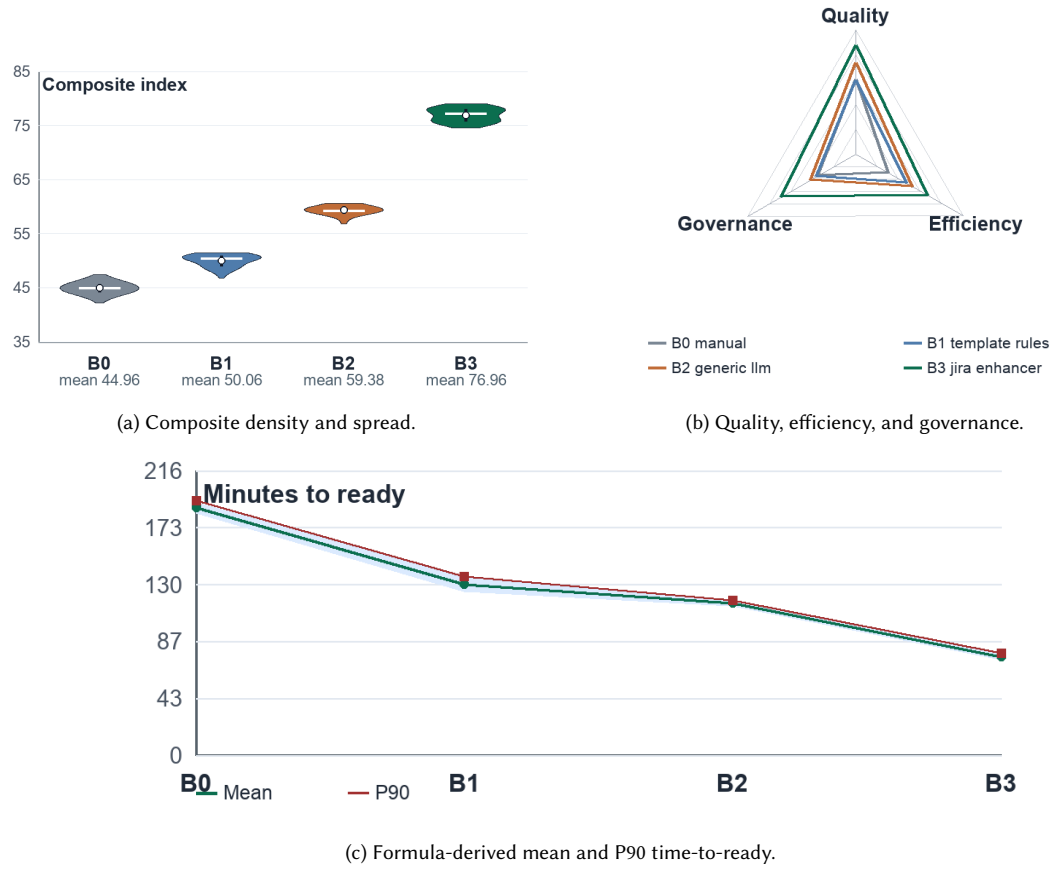


Fig. 8. Three diagnostics from the 5,000-unit formula stress test. Panel (a) shows density, interquartile range, median, and mean. Panel (b) shows the mean component indices. Panel (c) shows the generated mean and P90 time-to-ready. Its shaded band is ± 1 standard deviation, not a confidence interval. These panels audit the scoring model. They do not report observed comparative performance or measured speedups.

Figure 8b separates the composite index into quality, efficiency, and governance. The generated B2 profile records 73.76, 52.33, and 41.97 on these dimensions. The generated B3 profile records 88.18, 66.97, and 68.76. The largest modeled gap is in governance. This result explains how the declared B3 formula produces a higher composite score. It does not measure a live B2 or B3 execution.

Figure 8c shows the generated time-to-ready values. The mean values for B0–B3 are 187.88, 130.04, 115.37, and 74.49 minutes. The corresponding P90 values are 193.14, 135.96, 117.28, and 77.11 minutes. The shaded band shows one standard deviation of the generated values. It is not a confidence interval. The horizontal axis lists four categorical profiles. It is not a chronological trajectory. Therefore, the plot is a sensitivity check of the timing formulas. It is not evidence of measured time savings or speedup.

All three panels audit deterministic outputs of the declared formulas. The 5,000 rows per profile are seeded perturbations of reused runtime-derived units. They are not 5,000 independent epics or executed treatments. The plots also do

not replace the blinded matched comparison. In that comparison, the B3–B2 package-score difference was 0.10 points, with a 95% confidence interval of $[-0.13, 0.36]$ and a Holm-adjusted p -value of 0.485.

6.5 Generator-Separation Diagnostics

We define the primary inferential target on composite index:

$$H_0 : \mu_{B0} = \mu_{B1} = \mu_{B2} = \mu_{B3}, \quad (57)$$

$$H_1 : \exists m_1, m_2 \in \{B0, B1, B2, B3\} \text{ such that } \mu_{m_1} \neq \mu_{m_2}. \quad (58)$$

Based on the generated dataset ($n = 20,000$, with $n = 5,000$ per profile), we calculate the following implementation diagnostics: the Kruskal–Wallis test statistic and one-way ANOVA:

- Kruskal–Wallis $H_{KW} = 18744.32$, $p < 10^{-300}$.
- One-way ANOVA $F_{ANOVA} = 930378.49$, $p < 10^{-300}$.

Both tests reject H_0 for the generated data. However, the rows are deterministic perturbations of a much smaller base dataset, and all four profiles have the same units. Thus, the calculated p -values measure the degree of separation produced by the algorithm itself; they provide no grounds for population inference and do not prove superiority over real-world samples.

For completeness, the generated dataset also provides Mann–Whitney U_{MW} , Holm correction, bootstrap 95% confidence intervals, and Cliff’s δ_{Cliff} [15, 16, 23, 43]. These measures remain here to enable auditing of the existing generator rather than turn deterministic pseudo-replications into empirical observations. Pairwise hypothesis tests for matched-input experiments should use paired or hierarchical analysis, with effect sizes [61] reported separately from significance tests.

Given samples $X = \{x_i\}_{i=1}^{n_x}$ and $Y = \{y_j\}_{j=1}^{n_y}$, Cliff’s effect size is:

$$\delta_{Cliff} = \frac{\#\{(i, j) : x_i > y_j\} - \#\{(i, j) : x_i < y_j\}}{n_x n_y}. \quad (59)$$

The confidence intervals for bootstrap [16] are produced by resampling X and Y with replacement, calculating $\Delta = \bar{X} - \bar{Y}$ for every replicate, and taking the 2.5th and 97.5th percentiles as confidence interval endpoints. Holm adjustment [23] orders raw p -values for pairwise tests and tests for the hypothesis that the i th ordered value is less than $\alpha_{sig}/(M_{cmp} - i + 1)$ for a family of M_{cmp} comparisons. This adjustment is more conservative compared to raw pairwise testing and supplements false-discovery-rate control [8] for large comparison families.

Table 12. Pairwise separation diagnostics of formula-computed composite indices ($n = 5,000$ generated units per profile). These numbers are diagnostics of the generated model and not population-level tests of the baselines.

Comparison	$\Delta\mu$	95% CI (low)	95% CI (high)	Cliff’s δ	Holm- p	Sig.
B3 vs B0	32.01	31.96	32.05	1.000	<1e-300	yes
B3 vs B1	26.90	26.85	26.94	1.000	<1e-300	yes
B3 vs B2	17.58	17.54	17.62	1.000	<1e-300	yes

Table 13. Cross-input sensitivity comparison of the internal generated units and 300 SWE-bench Lite issue texts. Δ to B3 is calculated from formula-derived composite values within each input set.

Dataset	Method	N	Quality	Efficiency	Governance	Composite	Δ to B3
Internal 5,000-unit run	B0	5000	60.17	30.28	35.18	44.96	+32.01
Internal 5,000-unit run	B1	5000	59.66	47.01	36.45	50.06	+26.90
Internal 5,000-unit run	B2	5000	73.76	52.33	41.97	59.38	+17.58
Internal 5,000-unit run	B3	5000	88.18	66.97	68.76	76.96	+0.00
SWE-bench Lite test	B0	300	60.39	30.62	35.21	45.16	+32.73
SWE-bench Lite test	B1	300	57.72	48.20	35.34	49.27	+28.62
SWE-bench Lite test	B2	300	79.36	48.39	42.79	60.93	+16.96
SWE-bench Lite test	B3	300	89.92	66.65	69.72	77.89	+0.00

These diagnostic results confirm that the implemented algorithm generates distinct profiles as was promised. They do not answer RQ4 empirically; that requires actual baseline executions and independent outcome assessment.

6.6 Public Issue-Text Sensitivity Check (SWE-bench Lite)

Using the SWE-bench Lite public test split ($n = 300$) [28], the analysis extracts problem text, hints, and test identifiers; derives lexical complexity, confidence, and readiness features; and applies the same B0–B3 formulas. It does not ask any method to generate a patch, execute repository tests, or measure issue resolution. The following tables are therefore a public issue-text transfer check for the scoring generator, not SWE-bench performance results.

Table 13 shows that the formulas produce the same ranking on both input sets, with $B3 > B2 > B1 > B0$. Formula-derived B3–B2 differences are +17.58 internally and +16.96 on public issue text. This agreement indicates stability of the scoring assumptions under a change in text source; it does not demonstrate cross-dataset method effectiveness.

Figure 9 visualizes the same formula behavior. Similar polygon shapes show that the scoring generator responds consistently to the two text sources.

The scoring model assigns mean composites of 60.93 to B2 and 77.89 to B3, a difference of +16.96 points (27.84% relative to B2). This is a sensitivity delta, not a measured before/after improvement.

All formula-derived B3 profiles are separated from the other profiles after Holm correction ($p < 10^{-90}$ in this generated sample), with large Cliff’s δ_{Cliff} . As above, these statistics diagnose generator separation and do not strengthen external validity of the deployed method.

Relationship to the executed matched comparison. The matched study carries out the prespecified primary B2–B3 comparison and the secondary B1–B3 comparison on common inputs. B0 is absent because no independently authored manual package was available for the same tasks. Although the generated sensitivity model encodes a large B3–B2 separation, the observed composite difference is not statistically distinguishable from zero. This is an important negative finding. Formula-derived rankings can stress-test the implementation, but they cannot replace blinded human assessment. A future external replication should add a genuine manual baseline, new domains, a cost-matched B2 condition, and downstream delivery outcomes.

6.7 Artifact and Confidentiality Boundary

The review artifact retains deterministic analysis scripts, de-identified telemetry for the 70 story endpoints, a de-identified audit of the latest-wave payloads, coded ratings from both artifact assessments, aggregate matched-test

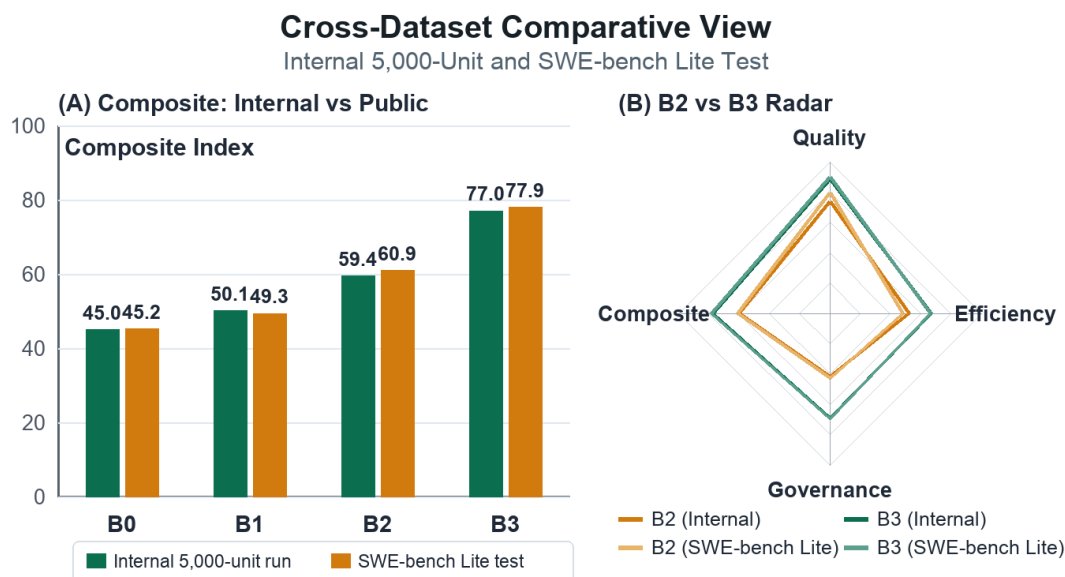


Fig. 9. Cross-input sensitivity view. Panel (A) reports formula-derived B0–B3 composite scores for internal generated units and public issue text. Panel (B) overlays the corresponding B2/B3 profiles. No patches or benchmark tests are executed.

tables, fault-injection trial records, seeds, and checksums. The shareable data exclude issue and epic keys, internal URLs, account names, email addresses, and timestamps. Raw Jira and Confluence records, the sanitized matched-input packets, and the private method-allocation key remain access-controlled because the source requirements are confidential enterprise material. We did not collect reviewer names, demographic variables, sensitive attributes, or other personal identifiers. The study also does not measure trust, workload, satisfaction, or individual performance.

Table 14. Formula-derived SWE-bench Lite issue-text sensitivity results: all-method profiles, the B2–B3 difference, and pairwise separation diagnostics. These values are scoring-model outputs, not issue-resolution benchmark scores or executed patch results.

Panel A: All-method profile summary						
Method	Quality	Efficiency	Governance	Composite	Time-to-ready (min)	
B0	60.39	30.62	35.21	45.16	187.03	
B1	57.72	48.20	35.34	49.27	122.78	
B2	79.36	48.39	42.79	60.93	139.62	
B3	89.92	66.65	69.72	77.89	74.63	
Panel B: B2–B3 difference						
Comparison		Before mean	After mean	Absolute gain	Relative gain (%)	
B2 → B3 on SWE-bench Lite		60.93	77.89	16.96	27.84	
Panel C: Pairwise separation diagnostics						
Comparison	$\Delta\mu$	95% CI (low)	95% CI (high)	Cliff’s δ	Holm- p	
B3 vs B2	16.96	16.85	17.07	1.000	3.16e-99	
B3 vs B1	28.62	28.48	28.76	1.000	2.11e-99	
B3 vs B0	32.73	32.51	32.95	1.000	1.05e-99	

7 DISCUSSION: POST-EVALUATION ANALYSIS

7.1 Reliability and Error Handling

The system classifies warning codes, such as external authentication requirements and MCP unavailability, and propagates them to run- and story-level review records. This supports graceful degradation:

$$\text{service_mode} \in \{\text{full-context}, \text{reduced-context}\}. \quad (60)$$

Full-context mode is used for cases with external evidence; *reduced-context* mode is applied when only issue-local evidence is available.

Reduced-context mode is designed to remain useful instead of failing softly. It still provides structured drafts and warning metadata, which allows reviews to continue in an informed way.

In reliability engineering, the “graceful capability collapse” pattern is applied instead of binary failure. For example, if a high-value evidence source such as Confluence is offline, the system retains decomposition capability by using the available issue records and marking the missing evidence categories. Available issue records prevent planning deadlock, while metadata preservation maintains the meaning of the confidence score.

Transient failures are retried using exponential backoff and jitter, while terminal errors are reported with machine-actionable error codes. This approach enables the orchestration layer to manage retries automatically without concealing permanent integration problems.

The user-driven workflow preserves collaboration state between steps when failures occur. If story creation times out or partially fails, the selected stories remain available in the console. The user may examine partial generation in Jira

and resume work without regeneration of the decision set. This ability is crucial for human-AI collaboration because recovery cannot be just a back-end feature; it must be observable to a human operator performing retry, revision, or termination decisions.

7.2 Positioning and Novelty Scope

The evidence blocks can confirm different claims: 70 authenticated endpoints demonstrate feasibility of a successful path; independent ratings cover 69 full-text stories and 14 epic packages; five reviewers compared 42 B1, B2, and B3 packages with 210 stories; fault injection demonstrates safety of writeback; and replaying, 5,000-unit, and public issue-text blocks verify declared numerical behavior. The study does not measure cross-project and longitudinal delivery benefits, which remain important in light of unreliability, human values, and unavailability of standardized evaluation of AI in software engineering [1, 14, 17].

Prompt learning is another part of B3's novelty. B3 does not use an unrestricted prompt loop. It represents prompt packs as bounded policy arms. A contextual bandit selects only from the arms that pass the governance checks [5, 35]. The selected arm, refinement result, and posterior update are recorded for auditing, as shown in Table 3 and Algorithm 2. This design makes prompt selection adaptive without giving the learning layer authority to modify Jira. Final mutation still requires validation, policy checks, and human approval. However, the matched experiment evaluated the complete B3 pipeline with a frozen pre-study policy. It did not isolate the effect of the RL component. Therefore, the results support the full B3 workflow, not a causal claim that prompt learning alone produced the observed advantage.

B3 controls learning, confidence, governance, approval, recovery, and conflict-free writeback. The score of B3's blinded artifact did not differ significantly from B2's, but B3 took less assessment time, had a descriptively better revision profile, and was the only one that could proceed through a source-bound mutation pathway tested under stale state, retry, draft drift, policy drift, and target drift. This evidence supports the bounded advantage of end-to-end workflows.

8 LESSONS LEARNED

The study produced six practical lessons. Each lesson is limited to the evidence that supports it.

- (1) **Different evaluations answer different questions.** The study uses four types of evidence, as summarized in Table 4. First, the field trace contains 70 verified end states. These include 69 newly created Jira issues and one safe-duplicate resolution. The latest wave also includes 44 audited new payloads. Second, human reviewers assessed artifact quality. Two reviewers assessed 69 full-text stories and 14 epic packages. Five reviewers compared 42 matched B1, B2, and B3 packages containing 210 stories. Third, the fault-injection experiment tested mutation safety across 72,000 scenario-balanced observations. Finally, offline replay and formula-based sensitivity analyses remain diagnostic evidence. These evaluations have different goals and units. Therefore, their results are reported separately and are not combined into one overall result.
- (2) **B3 automates refinement that B2 leaves outside the pipeline.** B2 produced one generic draft for each epic and then stopped. It had no critic pass, confidence feedback, retry loop, or prompt-policy selection. Under the evaluated B2 design, any further refinement would require human direction or another controller. B3 used its bounded critic loop and RL-guided prompt-policy selection to perform 159 targeted refinement attempts across 70 stories. These attempts were selected and applied automatically. The matched comparison did not count how many follow-up prompts a person would need for B2. It therefore cannot report an exact prompt saving. However, reviewers assessed B3 packages 1.02 minutes faster and assigned 5 Major revision decisions, compared

with 14 for B2, as shown in Table 9. The aggregate artifact-score difference was not statistically significant. B3 was also much stronger than the deterministic B1 baseline. The supported advantage is automated, governed refinement with lower observed reviewer burden. It is not evidence of better prose or lower compute cost. In operational use, human input remains necessary for missing domain decisions and final approval. Approval, recovery, and safe writeback require separate evaluation.

- (3) **Human approval needs a technical boundary.** Approval alone does not prevent a stale or duplicate mutation. The approved draft must remain linked to the source state that the reviewer examined. The source-bound transaction envelope provides this link. In the fault-injection experiment, it blocked all injected unsafe mutations and allowed all eligible writes, as reported in Table 10. Human authority is stronger when the writeback contract enforces the approval decision.
- (4) **Missing evidence should remain visible.** The system can continue in reduced-context mode when external evidence is unavailable. It must not hide the missing source. Warning codes and confidence provenance help the reviewer understand this limitation. A draft can remain useful without being presented as fully supported.
- (5) **Recovery should preserve human decisions.** Generation and writeback can fail at different stages. A useful workflow should keep the selected stories and partial outcomes visible. The reviewer should be able to revise, retry, or stop. Silent regeneration would weaken oversight and auditability.
- (6) **Adaptive claims need complete trajectories and cost evidence.** The live data retain terminal endpoints but not every refinement attempt. They cannot show a complete learning trajectory. B3 also used 173 model calls, compared with 14 for B2. Faster package assessment is therefore not evidence of lower total compute cost. Future studies should retain every attempt and report cost-normalized outcomes.

Together, these lessons support a bounded conclusion. B3 provides a governed path from evidence to an approved mutation under the evaluated conditions. The study does not prove superior prose, delivery quality, productivity, compute efficiency, or generalizability across projects.

9 IMPLICATIONS, LIMITATIONS, AND THREATS TO VALIDITY

9.1 Future Extensions

Future work can extend Jira Enhancer in five areas. First, multi-objective reinforcement learning can balance confidence with model cost. Second, project velocity can support story-point estimation. Third, active learning can identify the stories that need human review first. Fourth, multilingual prompts can support teams that work in different languages. Finally, a dependency graph can connect Jira issues, documents, code, and incidents. This graph can help the system find related work and identify critical paths before decomposing an epic. All these extensions would remain subject to the existing policy and audit controls.

9.2 Limitations and Validity Threats

Internal validity: this concerns whether the observed differences come from the evaluated methods rather than the study design. We used the same inputs, neutral formatting, independent randomization, and blinding to reduce bias [65]. However, the workflows were not matched for compute. For B1, the static completion rule filled slots for 24 of 70 stories. B2 used 14 model calls, and B3 used 173. Reviewers 1 and 2 had previously seen B3-only material. We therefore excluded them from the primary analysis. The generated-unit analysis also uses formulas rather than executed treatments. It cannot estimate a treatment effect [56].

Construct validity: this concerns whether the measures represent the concepts claimed in the study. Confidence is produced and optimized by the same pipeline. It is therefore not an independent measure of quality. Human approval records a workflow decision, not later delivery success. Blinded reviewer ratings measure visible content quality, but they do not measure confidence calibration. Recorded assessment time is not total planning time, and the edit counts are estimates. Reviewers could not see the governance metadata. Therefore, the artifact score does not measure policy or writeback value. The composite-score weights are declared, but ECE and the Brier score cannot be estimated without downstream outcome labels.

External validity: this concerns whether the findings can be applied to other settings. The 70 endpoints come from 14 epics, two deployment waves, and one organization. They represent successful paths only. The data contain no rejected, abandoned, below-threshold, or conflict endpoint. The matched study uses the same portfolio and has no independently authored manual baseline. The public issue-text analysis also does not execute the methods. Claims about transfer therefore require new projects, domains, and practitioners. This is especially important for context-dependent socio-technical tools [56, 57].

Conclusion validity: this concerns the strength and precision of the conclusions. The endpoints are nested within epics, the two waves begin with different score distributions, and all terminal decisions succeed. The epic-level exact and cluster-resampled analyses account for this nesting. However, the 14 epic clusters provide limited precision. The B3–B2 interval still permits modest effects in either direction. The ratings showed high inter-rater agreement. The distinct comments indicate that reviewers made independent judgments. However, each reviewer completed only four repetitions. We did not record reviewer experience bands or ask reviewers to guess which method produced each package. The disposition results are descriptive. The fault-injection rates are scenario-balanced and should not be treated as estimates of production prevalence.

Reference currency: research on human–AI collaboration in software engineering is developing quickly. The related literature should therefore be reviewed and updated periodically [64].

Mitigation: future studies should use independently prepared manual packages and compute-matched B2 and B3 configurations. The blinded study should be repeated on new portfolios, and reviewer expertise levels should be recorded. Ablation tests should disable the critic, retrieval, validation, and policy layers one at a time. Future work should also test sensitivity to the selected weights and assess the practical significance of any change. Finally, later studies should observe edits, failures, rework, and cycle-time variation across multiple releases.

10 CONCLUSION

Jira Enhancer is a mathematically specified, executable algorithm for governed AI-assisted planning. It generates reviewable story proposals from underspecified epics, maintains confidence and provenance of evidence, insists on human approval, and governs mutation through source-bound validation and idempotency. Two authenticated deployment waves demonstrate successful-path feasibility, while controlled fault injection tests protection against stale-state and duplicate mutations. Evaluation of authenticated outputs indicates strong scope fit, coverage, non-redundancy, and decomposition consistency, while also identifying where human refinement was required, particularly for acceptance criteria, dependencies, risks, and readiness details. The later matched study with five reviewers provides a more discriminating result. B3 was clearly stronger than deterministic templating. Compared with the text-only B2 condition, the composite artifact evaluation showed no discernible mean difference; however, B3 required 1.02 minutes less review time, received 5 rather than 14 Major revision decisions across 42 primary evaluations, and offered governance and mutation-protection capabilities beyond those of B2, and blocked every injected unsafe mutation while

allowing every eligible write within the source-bound transaction envelope. Under the evaluated conditions, these findings identify B3 as the stronger end-to-end governed planning workflow, although it required substantially greater exposure to model calls. This does not mean that orchestration produces generally better prose. On the contrary, AI can engage in planning without silently taking control of the tracker by having its proposals tied to evidence, reviewers being able to review and fix failures, and mutation being gate controlled. An effective orchestration claim also needs delivery outcomes downstream, cost-normalized performance measures, failure cases in the field, and generalization across domains.

REFERENCES

- [1] Iftekhar Ahmed, Aldeida Aleti, Haipeng Cai, Alexander Chatzigeorgiou, Pinjia He, Xing Hu, Mauro Pezzè, Denys Poshyvanyk, and Xin Xia. 2025. Artificial Intelligence for Software Engineering: The Journey So Far and the Road Ahead. *ACM Transactions on Software Engineering and Methodology* 34, 5, Article 119 (2025), 27 pages. <https://doi.org/10.1145/3719006>
- [2] Saleema Amershi, Dan Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, Penny Collisson, Jina Suh, Shamsi Iqbal, Paul N. Bennett, Kori Inkpen, Jaime Teevan, Ruth Kikin-Gil, and Eric Horvitz. 2019. Guidelines for Human-AI Interaction. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, 1–13.
- [3] John Anvik, Lyndon Hiew, and Gail C. Murphy. 2006. Who Should Fix This Bug?. In *Proceedings of the International Conference on Software Engineering*. Association for Computing Machinery, New York, NY, USA, 361–370.
- [4] Akari Asai, Sewon Min, Zexuan Zhong, Danqi Chen, Hannaneh Hajishirzi, Wen-tau Yih, Yejin Choi, Mike Lewis, and Luke Zettlemoyer. 2024. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In *International Conference on Learning Representations (ICLR)*.
- [5] Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. 2002. Finite-Time Analysis of the Multiarmed Bandit Problem. *Machine Learning* 47, 2–3 (2002), 235–256. <https://doi.org/10.1023/A:1013689704352>
- [6] Gagan Bansal, Besmira Nushi, Ece Kamar, Walter S. Lasecki, Daniel S. Weld, and Eric Horvitz. 2019. Beyond Accuracy: The Role of Mental Models in Human-AI Team Performance. In *Proceedings of the AAAI Conference on Human Computation and Crowdsourcing*. AAAI Press, Palo Alto, CA, USA, 2–11.
- [7] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2023. Grounded Copilot: How Programmers Interact with Code-Generating Models. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1 (2023), 85–111.
- [8] Yoav Benjamini and Yosef Hochberg. 1995. Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing. *Journal of the Royal Statistical Society: Series B* 57, 1 (1995), 289–300.
- [9] Philip A. Bernstein and Eric Newcomer. 2009. *Principles of Transaction Processing* (2 ed.). Morgan Kaufmann, Burlington, MA, USA.
- [10] Nicolas Bettenburg, Sascha Just, Adrian Schröter, Cathrin Weiss, Rahul Premraj, and Thomas Zimmermann. 2008. What Makes a Good Bug Report?. In *Proceedings of the ACM SIGSOFT International Symposium on Foundations of Software Engineering*. Association for Computing Machinery, New York, NY, USA, 308–318.
- [11] Glenn W. Brier. 1950. Verification of Forecasts Expressed in Terms of Probability. *Monthly Weather Review* 78, 1 (1950), 1–3. [https://doi.org/10.1175/1520-0493\(1950\)078<0001:VOFEIT>2.0.CO;2](https://doi.org/10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2)
- [12] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [13] Lan Cao and Balasubramaniam Ramesh. 2008. Agile Requirements Engineering Practices: An Empirical Study. *IEEE Software* 25, 1 (2008), 60–67.
- [14] Haowei Cheng, Jati H. Husen, Yijun Lu, Teeradaj Racharak, Nobukazu Yoshioka, Naoyasu Ubayashi, and Hironori Washizaki. 2024. Generative AI for Requirements Engineering: A Systematic Literature Review. arXiv:2409.06741.
- [15] Norman Cliff. 1993. Dominance Statistics: Ordinal Analyses to Answer Ordinal Questions. *Psychological Bulletin* 114, 3 (1993), 494–509. <https://doi.org/10.1037/0033-2909.114.3.494>
- [16] Bradley Efron. 1979. Bootstrap Methods: Another Look at the Jackknife. *The Annals of Statistics* 7, 1 (1979), 1–26. <https://doi.org/10.1214/aos/1176344552>
- [17] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang. 2023. Large Language Models for Software Engineering: Survey and Open Problems. arXiv:2310.03533.
- [18] Henning Femmer, Daniel Méndez Fernández, Stefan Wagner, and Sebastian Eder. 2017. Rapid Quality Assurance with Requirements Smells. *Journal of Systems and Software* 123 (2017), 190–213.
- [19] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. 2024. CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. In *The Twelfth International Conference on Learning Representations*. https://proceedings.iclr.cc/paper_files/paper/2024/hash/fe126561bbf9d4467dbb8d27334b8fe-Abstract-Conference.html

- [20] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. On Calibration of Modern Neural Networks. In *Proceedings of the 34th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 70)*. PMLR, Sydney, NSW, Australia, 1321–1330.
- [21] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. 2020. REALM: Retrieval-Augmented Language Model Pre-Training. In *International Conference on Machine Learning (ICML)*.
- [22] Rashina Hoda. 2026. Toward Agentic Software Engineering Beyond Code: Framing Vision, Values, and Vocabulary. In *Proceedings of the 2026 International Workshop on Agentic Engineering*. Association for Computing Machinery, New York, NY, USA, 181–185. <https://doi.org/10.1145/3786167.3788422>
- [23] Sture Holm. 1979. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6, 2 (1979), 65–70. <https://www.jstor.org/stable/4615733>
- [24] Irum Inayat, Siti Salwah Salim, Sabrina Marczak, Maya Daneva, and Shahaboddin Shamshirband. 2015. A Systematic Literature Review on Agile Requirements Engineering Practices and Challenges. *Computers in Human Behavior* 51 (2015), 915–929.
- [25] ISO/IEC/IEEE. 2018. *ISO/IEC/IEEE 29148:2018 Systems and Software Engineering—Life Cycle Processes—Requirements Engineering*. ISO/IEC/IEEE.
- [26] Gaeul Jeong, Sunghun Kim, and Thomas Zimmermann. 2009. Improving Bug Triage with Bug Tossing Graphs. In *Proceedings of the Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering*. Association for Computing Machinery, New York, NY, USA, 111–120.
- [27] Zhengbao Jiang, Jun Araki, Haibo Ding, and Graham Neubig. 2021. How Can We Know When Language Models Know? On the Calibration of Language Models for Question Answering. *Transactions of the Association for Computational Linguistics* 9 (2021), 962–977. https://doi.org/10.1162/tacl_a_00407
- [28] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations (ICLR)*.
- [29] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- [30] Omar Khattab and Matei Zaharia. 2020. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In *International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*.
- [31] Barbara A. Kitchenham, Shari Lawrence Pfleeger, Lesley M. Pickard, Peter W. Jones, David C. Hoaglin, Khaled El Emam, and Jarrett Rosenberg. 2002. Preliminary Guidelines for Empirical Research in Software Engineering. *IEEE Transactions on Software Engineering* 28, 8 (2002), 721–734.
- [32] Andrew J. Ko, Brad A. Myers, and Duen Horng Chau. 2006. A Linguistic Analysis of How People Describe Software Problems. In *Proceedings of the IEEE Symposium on Visual Languages and Human-Centric Computing*. IEEE Computer Society, Los Alamitos, CA, USA, 127–134.
- [33] John D. Lee and Katrina A. See. 2004. Trust in Automation: Designing for Appropriate Reliance. *Human Factors* 46, 1 (2004), 50–80.
- [34] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [35] Lihong Li, Wei Chu, John Langford, and Robert E. Schapire. 2010. A Contextual-Bandit Approach to Personalized News Article Recommendation. In *Proceedings of the 19th International Conference on World Wide Web*. Association for Computing Machinery, New York, NY, USA, 661–670. <https://doi.org/10.1145/1772690.1772758>
- [36] Stephanie Lin, Jacob Hilton, and Owain Evans. 2022. TruthfulQA: Measuring How Models Mimic Human Falsehoods. In *Annual Meeting of the Association for Computational Linguistics (ACL)*.
- [37] John D. C. Little. 1961. A Proof for the Queuing Formula: $L = \lambda W$. *Operations Research* 9, 3 (1961), 383–387. <https://doi.org/10.1287/opre.9.3.383>
- [38] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173.
- [39] Xiao Liu, Kaifeng Yu, Dan Zhang, Yao Wang, Diyi Song, Tianqi Chen, and et al. 2024. AgentBench: Evaluating LLMs as Agents. In *International Conference on Learning Representations (ICLR)*.
- [40] Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Haoping Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. 2025. ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities. In *Findings of the Association for Computational Linguistics: NAACL 2025*. Association for Computational Linguistics, Albuquerque, New Mexico, 1160–1183. <https://doi.org/10.18653/v1/2025.findings-naacl.65>
- [41] Garm Lucassen, Fabiano Dalpiaz, Jan Martijn E. M. van der Werf, and Sjaak Brinkkemper. 2016. Improving Agile Requirements: The Quality User Story Framework and Tool. *Requirements Engineering* 21, 3 (2016), 383–403.
- [42] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc. <https://doi.org/10.52202/075280-2019>
- [43] Henry B. Mann and Donald R. Whitney. 1947. On a Test of Whether One of Two Random Variables Is Stochastically Larger than the Other. *The Annals of Mathematical Statistics* 18, 1 (1947), 50–60. <https://doi.org/10.1214/aoms/1177730491>
- [44] Daniel Méndez Fernández and Stefan Wagner. 2015. Naming the Pain in Requirements Engineering: Contemporary Problems, Causes, and Effects in Practice. *Information and Software Technology* 57 (2015), 575–590.

- [45] Luc Moreau and Paolo Missier. 2013. *PROV-DM: The PROV Data Model*. W3C Recommendation. World Wide Web Consortium. <https://www.w3.org/TR/prov-dm/>
- [46] Bashar Nuseibeh and Steve Easterbrook. 2000. Requirements Engineering: A Roadmap. In *Proceedings of the Conference on the Future of Software Engineering*. Association for Computing Machinery, New York, NY, USA, 35–46.
- [47] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. Training Language Models to Follow Instructions with Human Feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [48] Raja Parasuraman, Thomas B. Sheridan, and Christopher D. Wickens. 2000. A Model for Types and Levels of Human Interaction with Automation. *IEEE Transactions on Systems, Man, and Cybernetics - Part A: Systems and Humans* 30, 3 (2000), 286–297.
- [49] Abhik Roychoudhury, Corina S. Pasareanu, Michael Pradel, and Baishakhi Ray. 2025. AI Software Engineer: Programming with Trust. <https://doi.org/10.48550/arXiv.2502.13767> arXiv:2502.13767 [cs.SE]
- [50] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=GEcwtMk1uA>
- [51] Per Runeson and Martin Höst. 2009. Guidelines for Conducting and Reporting Case Study Research in Software Engineering. *Empirical Software Engineering* 14, 2 (2009), 131–164.
- [52] Daniel J. Russo, Benjamin Van Roy, Abbas Kazerouni, Ian Osband, and Zheng Wen. 2018. A Tutorial on Thompson Sampling. *Foundations and Trends in Machine Learning* 11, 1 (2018), 1–96. <https://doi.org/10.1561/22000000070>
- [53] Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. 2022. ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction. In *Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*.
- [54] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [55] Noah Shinn, Federico Cassano, Bailin Labash, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [56] Klaas-Jan Stol and Brian Fitzgerald. 2018. The ABC of Software Engineering Research. *ACM Transactions on Software Engineering and Methodology* 27, 3, Article 11 (2018), 51 pages.
- [57] Margaret-Anne Storey, Neil A. Ernst, Courtney Williams, and Eirini Kalliamvakou. 2020. The Who, What, How of Software Engineering Research: A Socio-Technical Framework. *Empirical Software Engineering* 25, 5 (2020), 4097–4129.
- [58] Richard S. Sutton and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction* (2 ed.). The MIT Press, Cambridge, MA, USA. <https://mitpress.mit.edu/9780262039246/reinforcement-learning/>
- [59] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, 1–7.
- [60] Wil M. P. van der Aalst and Kees M. van Hee. 2004. *Workflow Management: Models, Methods, and Systems*. The MIT Press, Cambridge, MA, USA. <https://mitpress.mit.edu/9780262011891/workflow-management/>
- [61] András Vargha and Harold D. Delaney. 2000. A Critique and Improvement of the CL Common Language Effect Size Statistics of McGraw and Wong. *Journal of Educational and Behavioral Statistics* 25, 2 (2000), 101–132.
- [62] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [63] Frank Wilcoxon. 1945. Individual Comparisons by Ranking Methods. *Biometrics Bulletin* 1, 6 (1945), 80–83. <https://doi.org/10.2307/3001968>
- [64] Claes Wohlin. 2014. Guidelines for Snowballing in Systematic Literature Studies and a Replication in Software Engineering. In *Proceedings of the International Conference on Evaluation and Assessment in Software Engineering*. Association for Computing Machinery, New York, NY, USA, 1–10.
- [65] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. 2012. *Experimentation in Software Engineering*. Springer, Berlin, Germany.
- [66] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Steven Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Chi Wang, and et al. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. In *Conference on Language Modeling (COLM)*.
- [67] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [68] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.